GENERATED DOCUMENT

Business Requirements Document

Newsletter Builder

Version 0.1.0 · commit 9abc092

November 6, 2026

Generated from source code

Reverse-engineered from the codebase at `newsletter-builder` (a client-only React/TypeScript/Vite single-page app — no backend, no database, no login) plus its own `docs/PROGRESS.md` feature history, covering git history `0094008`…`9abc092` on branch `feature/6a-brief-generation`.

How this BRD is organized: it is split into 4 parts (39 sections total) plus this index and a verification log. Every claim in every part is **Fact** unless explicitly tagged **Inference** (a conclusion drawn from behavior, not stated verbatim) or **Not Found in Codebase** (looked for, confirmed absent). See `VERIFICATION_NOTES.md` for the complete, section-by-section list of every Inference and Not-Found item in this document.

Table of Contents

Part 1 — BRD-01.md

1. Executive Summary
2. Business Overview
3. Business Objectives
4. Stakeholders
5. User Roles
6. Functional Scope
7. Complete Business Process
8. Module Documentation
9. Feature Documentation
10. Screen Documentation

Part 2 — BRD-02.md

11. Navigation Flow
12. User Journey
13. Business Rules
14. Workflow States
15. Data Requirements
16. Business Data Dictionary
17. Validation Rules
18. Notifications
19. Reports
20. Dashboards

Part 3 — BRD-03.md

21. Integrations
22. External Systems

23. Security Requirements
24. Audit Trail
25. Non-Functional Requirements
26. Assumptions
27. Constraints
28. Risks
29. Gap Analysis
30. Future Enhancements

Part 4 — BRD-04.md

31. Acceptance Criteria
32. User Stories
33. Business Glossary
34. Frequently Asked Questions
35. Business Process Diagrams
36. End-to-End Business Scenario
37. Traceability Matrix
38. Production Readiness (Business Perspective)
39. Final Business Knowledge Validation

Verification Notes

Every Inference and Not-Found-in-Codebase item in this BRD, listed by section, with its reasoning — so a reviewer can check this document's honesty without re-reading the whole thing.

One-paragraph orientation

Newsletter Builder is a **single-user, browser-only** drag-and-drop email template builder: no server, no database, no login — every brand kit, saved email, image, and AI key lives in one browser's local storage. It produces email-client-safe HTML and plain text for a human to paste into their own ESP (SES, SendGrid, Mailchimp, HubSpot, …); it never sends anything itself. Two AI features (image generation and "generate an email from a brief") call OpenAI directly from the browser with a user-supplied API key, always behind a non-blocking quality-check engine and (for briefs) a human-review step. If a BRD section below reads "Not Found in Codebase" for something a typical enterprise BRD would expect (roles, SSO, approval chains, integrations, audit logs, dashboards), that is because this product genuinely does not have one — not because the research missed it.

Part 1 of 4 — Sections 1–10

> **Fact vs. Inference vs. Not Found** — every claim in this document is one of three things:
>
> - **Fact** — stated directly in code, comments, tests, or `docs/PROGRESS.md`, and is the default status of any sentence with no other tag.
> - **Inference:** — a conclusion drawn from code behavior but not stated verbatim anywhere; flagged inline.
> - **Not Found in Codebase** — explicitly looked for and absent; flagged inline rather than guessed at.

---

1. Executive Summary

What the application is. Newsletter Builder is a **browser-only, single-user, drag-and-drop email template builder** (React 19 + TypeScript + Vite + Tailwind CSS 4). It lets a non-technical marketing user assemble a company newsletter or marketing email visually — rows, columns, and content blocks — and export **email-client-safe HTML** (table-based layout, inline styles, Outlook conditional markup) plus a plain-text alternative, ready to paste into an Email Service Provider (ESP) such as SES, SendGrid, Mailchimp, or HubSpot. (Fact, `README.md`.)

There is **no server, no database, and no login**. Every byte of data — designs, brand kits, saved blocks, the image library, version history, AI provider keys, AI-generated image drafts — lives in the visiting browser's `localStorage` (~5 MB budget) or `IndexedDB`, and nothing syncs between machines or teammates except by manual JSON export/import. (Fact, README "Known limitations"; corroborated across every feature in `docs/PROGRESS.md`.)

Business purpose. `docs/PROGRESS.md` states the product is an **"internal tool for the Vee MARCOM team"** (Feature 1A) — i.e., it exists to let a marketing/communications team produce on-brand, email-safe HTML newsletters without a developer or an external design tool in the loop, and (as of Feature 6A) without necessarily writing the first draft of the copy by hand either.

Target organizations / users. A single organization's internal marketing/communications function (named explicitly: "Vee Technologies," seeded as the default brand). **Inference:** the architecture (per-browser brand kits, no roles, export/import as the sharing mechanism) suggests the product was built for a small team sharing a few brand kits, not a multi-tenant SaaS audience — this is an inference from the data model, not a stated target-market claim.

Business problems solved (each is a named feature below):

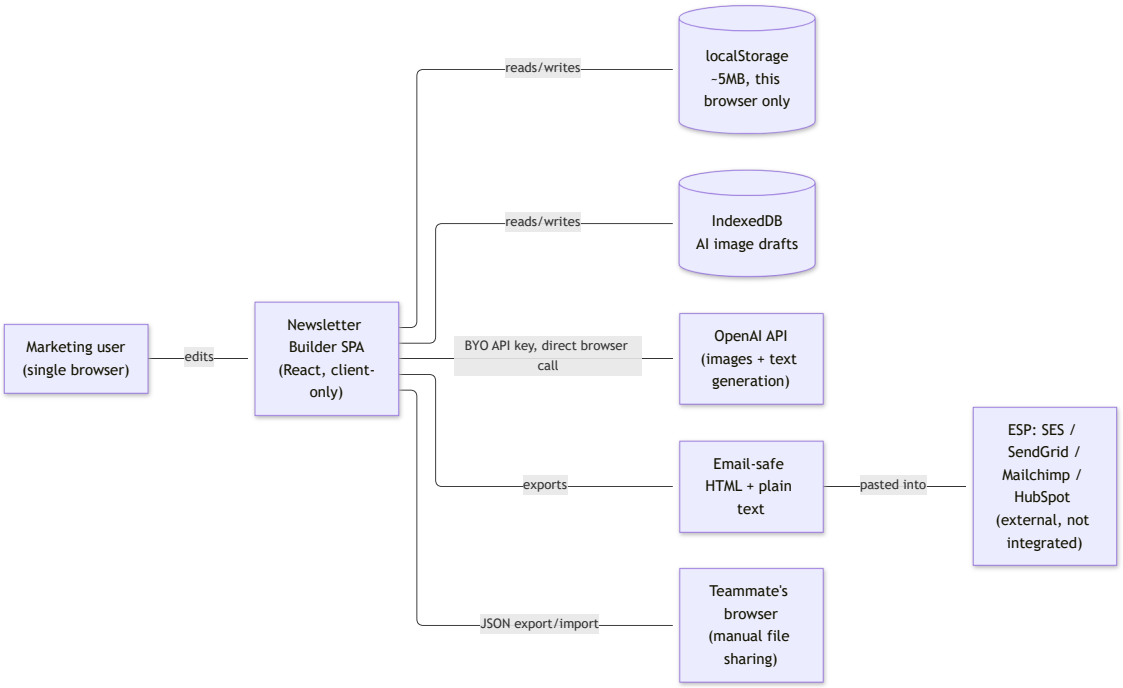
Problem	Feature	Solution
Newsletters require a designer/developer to hand-code HTML email	1A–1C, Canvas/renderer	WYSIWYG canvas; one renderer produces both the live preview and the export
Re-theming for a different brand means rebuilding the email	1A	Brand-token resolution: colors/fonts are tokens resolved at export time
Common sections (header, hero, article list, footer) rebuilt by hand every time	1B, 1C	Composite blocks + 8 ready-made presets
No visibility into whether an email will actually work in an inbox	3A (Checks), 2C-1 (image audit)	A non-blocking quality engine checking size, links, contrast, images, subject/preheader
Dark-mode inboxes show broken colors	3B	Per-brand dark palette, additive-only dark CSS
Starting a draft from scratch takes too long	6A	AI-assisted "generate an email from a brief," human-reviewed before anything is written to the design

Problem	Feature	Solution
Generating on-brand imagery requires a separate tool	5A	Bring-your-own-key AI image generation inside the editor
Settings/keys/brand management were scattered	7A	A single Settings hub; a local "profile" display name (not a login)
One AI model choice for everything, no fallback	6B	Per-task (text vs. image) model routing with a configurable fallback chain

Key capabilities (see §9 for full feature documentation): brand kits with light/dark palettes; 13 block types (9 atomic + 4 composite); 8 starter presets; drag-and-drop canvas with undo/redo; merge tags with 4 ESP output formats; UTM link tagging; plain-text generation; auto-save, version history, and a saved-blocks library; a URL-based image library; a non-blocking email-quality "Checks" engine; dark-mode export; AI image generation (OpenAI, bring-your-own-key); AI "generate email from a brief" with mandatory human review; per-task AI model routing with fallback chains; a Settings hub covering profile, AI, brands, email defaults, checks, and data backup/restore.

Expected business outcomes: faster newsletter production, consistent on-brand output across emails, fewer broken-in-the-inbox sends (via Checks), and a lower skill floor for producing a first draft (via AI brief generation) — all **Inference**, since the codebase states the mechanisms but not measured outcomes; no KPI/analytics instrumentation exists in the codebase (see §3).

High-level business architecture.



Value proposition. One tool that both builds the email (design-time) and proves it is inbox-safe (Checks, image audit, dark-mode additive guarantee) before a human ever pastes it into a sending tool — without requiring a backend, an account system, or ongoing hosting costs, at the cost of no cross-device sync and no true multi-user collaboration. (Fact + Inference as to "value proposition" framing.)

---

2. Business Overview

Why this application exists. `docs/PROGRESS.md` frames each feature around a concrete, named operational pain point rather than an abstract roadmap item. Representative quotes:

- 1A: brand/theme management was manual, with "no roles, membership or login" needed yet seams left for later.
- 2B: "closing a tab lost whatever was not explicitly saved, there was no way back once the undo stack was gone, and a CTA band built once had to be rebuilt by hand next time."
- 2C-1: "Images were the weakest part of the builder: every URL typed by hand... nothing telling you an image was broken, low-resolution or still preset stock art until it reached an inbox."
- 3A: "One engine that reads what is actually sent and says what an inbox will do with it."
- 6A: turning "a structured content brief into a fully-populated draft email... with a human review step before anything is written to the design."
- 7A: settings "scattered" across the top bar and a panel gear icon, with no storage-usage visibility or backup.

Current business challenges (addressed). Hand-coded email HTML is fragile across clients (Outlook/Gmail/Apple Mail render differently); marketing copy drafting is slow; brand consistency across campaigns is hard to enforce by hand; nothing warned a sender before a send that an email would be clipped, broken, or inaccessible.

Business opportunities. AI-assisted first-draft generation (6A) and AI image generation (5A) shorten the time from "we need an email about X" to a reviewable draft. Per-task model routing (6B) lets the team choose cost/quality trade-offs independently for text vs. image generation and add resilience via fallback chains.

Operational problems solved: version history and auto-save remove the "I lost my work" risk; the saved-blocks library removes "rebuild this CTA band every time"; the image library removes "which URL did we use for that hero image last time."

Organizational benefits: a non-developer can produce compliant, on-brand, multi-client-safe HTML without engineering involvement per email.

Departments involved. Fact: `docs/PROGRESS.md` names the **"Vee MARCOM team"** (marketing/communications) as the internal user base for Feature 1A. No other department, approval chain, or cross-team workflow is referenced anywhere in the codebase. **Not Found in Codebase:** any other named department (legal, sales, IT, compliance) interacting with the system.

Business scope.

In scope	Not Found in Codebase / explicitly out of scope
Visual design of a single HTML email at a time	Multi-email campaign scheduling
Brand kit management (colors, fonts, logos, footer, social, voice)	Sending the email (README: "No sending: export the HTML and send it through your ESP")
AI-assisted copy and image drafting (BYO API key)	Any backend, server, or managed AI billing — the user is billed directly by OpenAI

In scope	Not Found in Codebase / explicitly out of scope
Email-quality checks (size, links, contrast, images, subject/preheader)	Link-liveness checking beyond classification (checks.ts: "Links are classified, not fetched")
Local backup/restore and "clear all" data management	Cross-device sync; multi-user collaboration; login/accounts/roles
Merge tags, UTM tagging, plain-text generation at export time	A/B testing, send-time personalization engine, analytics on opens/clicks

Out-of-scope items are load-bearing facts, not gaps to be solved later in several cases — the README explicitly frames "no sending," "per-browser only," and "URL-only images (no upload/hosting)" as *deliberate, current-state* boundaries, not unfinished work. Several (hosting, sending) are flagged as candidate future work in §30.

---

3. Business Objectives

Primary objectives (Inference, derived from the pattern of shipped features — no objectives/OKR document exists in the codebase):

1. Let a non-developer produce an email-client-safe HTML newsletter without hand-coding.
2. Keep every email visually on-brand with minimal per-email effort (token-based brand resolution).
3. Catch inbox-breaking mistakes (size, broken links, missing alt text, poor contrast, placeholder art) before a human sends, without ever blocking the export.
4. Reduce first-draft effort via AI assistance for both copy (6A) and imagery (5A), always behind a human review/consent step.

Secondary objectives (Inference): 5. Protect against data loss (auto-save, version history, undo/redo). 6. Make reuse cheap (saved blocks, image library, presets). 7. Keep AI usage auditable and safe (keys never serialized into exportable state; consent gates; redacted error messages; no silent retries that could double-bill).

Business KPIs. Not Found in Codebase. There is no analytics, telemetry, or usage-metrics code anywhere in `src/` — no event tracking, no error-reporting SDK, no "time to first email" instrumentation. Any KPI claim here would be invented, so none is stated.

Success criteria. Not Found in Codebase as a formal acceptance document; however, each shipped feature in `docs/PROGRESS.md` carries its own test-suite-verified acceptance bar (e.g., Feature 3A: "the Export dialog's buttons are live with errors outstanding, by design and by test"). These are implementation-level acceptance tests, not business-level success metrics — see §31 for feature-by-feature Given/When/Then criteria derived from the same source.

Expected ROI. Not Found in Codebase. No cost model, pricing, or ROI calculation exists in the product. The only cost-relevant facts in-code are: (a) AI generation calls are billed directly by OpenAI to the user's own key — the app has no cost visibility or usage caps (Fact, `ai-features.md` research pass, confirmed by exhaustive grep); (b) the deliberate no-retry policy on AI calls exists specifically "to avoid double-billing" a request that may have already produced and been charged for.

Business value delivered (Inference, qualitative): faster time-to-draft; consistent brand application; fewer email-client rendering failures; reduced risk of sending with a broken unsubscribe link, placeholder art, or inaccessible contrast.

---

4. Stakeholders

Not Found in Codebase: a human stakeholder list, org chart, or RACI document. The stakeholders below are **Inference**, reconstructed from what the product's own seams (`createdBy`/`updatedBy`, permission functions, the single "Profile" display name) imply about who touches the system and how.

Stakeholder	Role	Responsibilities	Goals	Pain points (addressed)	System interaction	Decision authority
Marketing/Communications team member (the "Vee MARCOM team," per 1A)	Primary/sole end user	Build, review, and export newsletters; manage the team's brand kit(s); optionally generate drafts via AI	Produce on-brand, inbox-safe emails quickly	Manual HTML coding; broken-in-inbox sends; lost work; inconsistent branding	Full access to every screen — there is no restricted view	Full — the only user role that exists
Brand owner (Inference — same person or a nearby role, not a distinct system role)	Defines/maintains the brand kit(s) (colors, fonts, logo, footer, voice)	Keep the brand kit accurate and current	Brand consistency across every email	Rebuilding brand styling per email	Brands modal / Settings → Brands	Same technical authority as any user — `canEditBrand()` returns `true` unconditionally today
A future "admin"/"owner" role (Inference — a seam, not a shipped role)	Not implemented	N/A	N/A	N/A	`createdBy`/`updatedBy` fields and `canEditBrand`/`getAccessibleBrands` exist as **explicit future-proofing stubs** (1A)	None today — the functions always return "everything"/"true"
OpenAI (external, non-human "stakeholder" in the business-process sense)	AI text/image generation provider	Fulfill brief-generation and image-generation requests sent directly from the user's browser with the user's own API key	N/A (external vendor)	N/A	Called directly via `fetch` from the browser (no backend proxy)	None over the product; the product is designed never to send the user's API key anywhere but to this provider

Stakeholder matrix (RACI-style, Inference):

Activity	Marketing/MARCOM user
Build/edit an email	R, A
Manage brand kit	R, A
Approve an AI-generated draft before send	R, A (the "Mark as reviewed" gate — see §13)
Export/send the email	R, A (sending itself happens outside this tool)
Back up/restore local data	R, A

Because there is exactly one functional role, a traditional multi-party RACI collapses to a single column — this is a **Fact about the product's current scope**, not an omission in this document.

---

5. User Roles

Not Found in Codebase: any role beyond a single, undifferentiated user. `src/lib/user.ts`'s own header comment states the product has **"no login, no roles, an internal tool's substitute for an account."** (Fact.) There is no authentication, no session boundary between "users," and no server-side identity of any kind (confirmed by exhaustive grep for `auth|login|password|session` across `src/` — every hit is either AI-provider API-key handling or in-browser-tab AI session state, never an app login).

5.1 The one functional role: "Editor" (unnamed in code; Inference label for this document)

Attribute	Value
Business purpose	Produce and maintain the organization's email newsletters
Responsibilities	All of: content authoring, layout, brand management, AI-assisted drafting, quality review, export, local data backup
Permissions	Unrestricted — every screen, every action
Accessible modules	All (Canvas/editor, Brands, Templates, History, Checks, Image Library, Saved Blocks, AI Image panel, Brief generation, Settings hub in full)
Restricted modules	None
Approval authority	De facto total — "Mark as reviewed" on an AI-generated draft (§13) is the only formal gate in the product, and the same single role both triggers and clears it
Reporting responsibility	None found — no reporting/escalation feature exists
Business ownership	The display name set in **Settings → Profile** is stamped as `createdBy`/`updatedBy` on brands and saved emails (7A) — purely a local label, "never sent anywhere," not an access-control mechanism

5.2 "Identity" vs. "role" — a deliberate distinction

`getCurrentUser()` (`src/lib/user.ts`) returns a single free-text display name (default **"Guest"**, capped at 40 characters), used only for the `createdBy`/`updatedBy` attribution fields. It is explicitly **not** an authentication identity: it is not required, not unique, not verified, and not used anywhere to gate access. Two people sharing one browser profile would appear as whichever name was last typed into **Settings → Profile**.

5.3 Permissions matrix

Module / Action	Editor role
Edit canvas (blocks, rows, body settings)	✅
Manage brand kits (create/edit/duplicate/archive/import/export)	✅ (`canEditBrand()` always returns `true` — Fact, with the function itself documented in-code as a future permission seam)
View/switch brand list	✅ (`getAccessibleBrands()` always returns every active brand — same seam)
Generate AI images / AI email drafts	✅, gated only by: (a) a configured + tested API key, and (b) a one-time per-provider consent checkbox for text generation — **not** by any role
Approve ("Mark as reviewed") an AI brief draft	✅ — the only person who can do this is the same single-role user
Manage local data (backup/restore/clear all)	✅, with a typed-confirmation ("DELETE") safety gate on irreversible deletion — a UX safeguard, not a permission check

Not Found in Codebase: any viewer-only, approver-only, or admin-only role; any concept of "my emails" vs. "someone else's emails" (every saved email in the browser is visible and editable to whoever has the browser open).

---

6. Functional Scope

Each function below maps to one or more features documented fully in §9.

Function	Purpose	Business objective	Trigger	Inputs	Outputs	Dependencies	Users involved	Preconditions	Postconditions
Design a newsletter	Compose the email visually	Fast, accurate authoring	Opening the app / New / Templates	Block type choices, text, images, settings	An in-memory `Design` document	Canvas, LeftPanel, RightPanel, store	Editor	None	`design` mutated; autosave scheduled
Start from a preset	Skip a blank canvas	Faster time-to-draft	"New email" → Presets tab	Brand selection, preset choice	A fresh `Design` built from the chosen preset	`presets.ts`, `NewEmailModal`	Editor	—	New email becomes the active, unsaved-until-first-edit document
Generate from a brief	AI-assisted first draft	Faster time-to-draft, consistent tone	"New email" → "Generate from brief" tab	Topic, audience, goal, key points, tone, length, CTA, language, optional source text	A reviewable `BriefDraft` (not yet written to any design)	`create-from-brief.ts`, `slots.ts`, OpenAI text API	Editor	A connected text-generation key; one-time consent	Draft appears in `BriefReview`; nothing persisted until "Create email"
Manage brand kits	Central source of visual identity	Brand consistency	Brands modal / Settings → Brands	Colors, fonts, logos, footer, social links, voice guidance	A `Brand` record	`brand.ts`, `BrandsModal`	Editor	—	Brand available in every brand switcher across the app

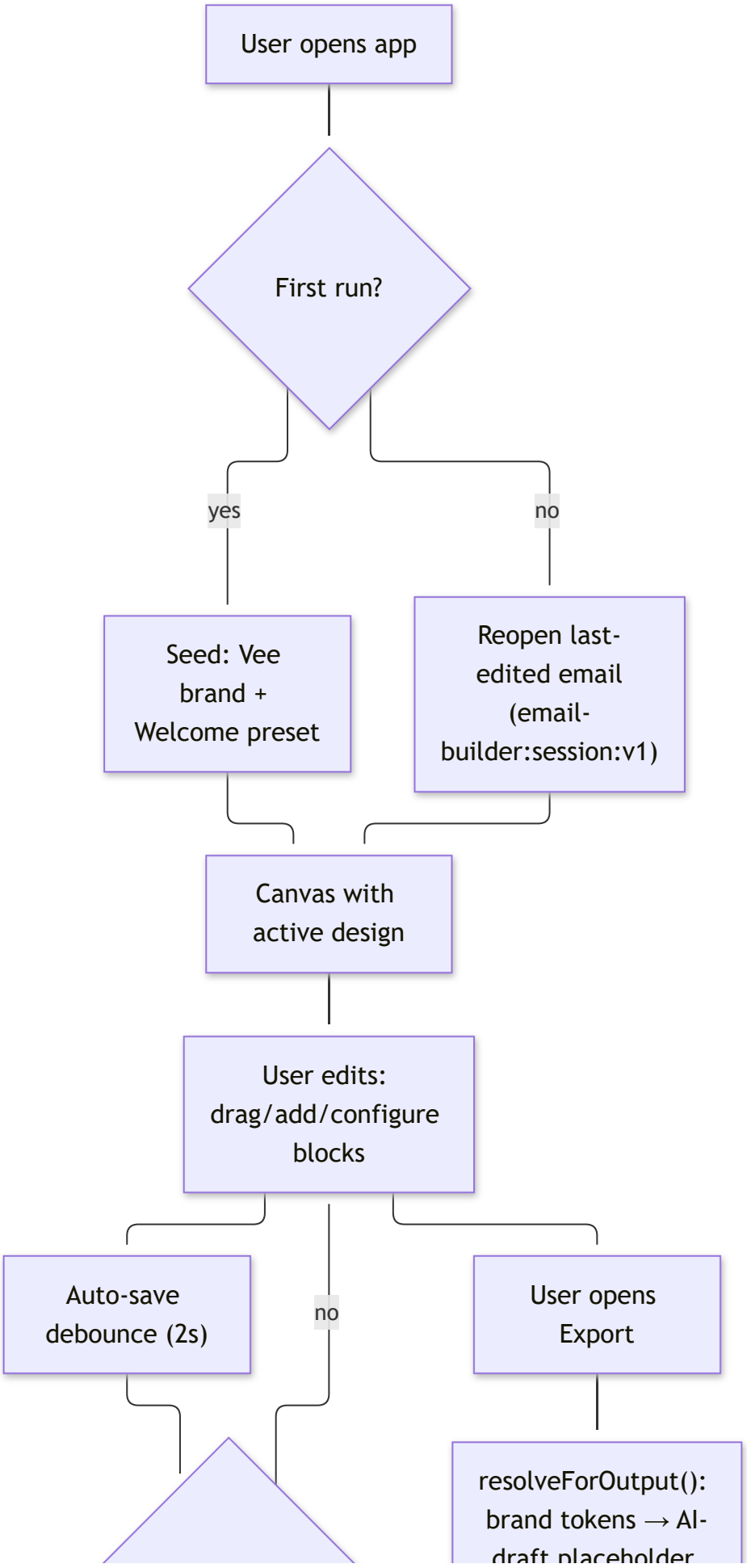
Function	Purpose	Business objective	Trigger	Inputs	Outputs	Dependencies	Users involved	Preconditions	Postconditions
Generate AI images	On-brand imagery without a separate tool	Faster asset creation	AI image panel (opened from an image field)	A text prompt, aspect, style preset, quality	A local image "draft" (IndexedDB bytes + an id)	`openai.ts` (image adapter), `drafts.ts`	Editor	A connected image-generation key	Draft usable on canvas; exports as placeholder until hosted externally
Run quality checks	Catch inbox-breaking issues before send	Fewer broken sends	Automatic (debounced) + on demand ("Check images load")	The resolved design + rendered HTML	A list of `Issue`s, grouped by severity	`checks.ts`, `contrast.ts`, `image-audit.ts`	Editor	—	Badge/drawer reflect current issues; never blocks export
Export the email	Produce the deliverable	Hand off to an ESP	"Export" button	The design, active brand, UTM/merge-tag settings	HTML and plain-text strings (Copy/Download)	`output.ts`, `render.ts`, `plaintext.ts`	Editor	—	Nothing persisted by export itself
Save / auto-save	Protect against data loss	No lost work	Any edit (2s debounce) or explicit "Save version"	The current design	A `SavedTemplate` record / `VersionRecord` snapshot	`storage.ts`, `versions.ts`	Editor	—	Record created/updated; top-bar status updated
Manage saved blocks	Reuse content across emails	Less rebuilding	"Save as block" on a selected block/row	A block or row payload	A `SavedBlock` library entry	`blocks-library.ts`	Editor	—	Entry available in the Saved tab across every email

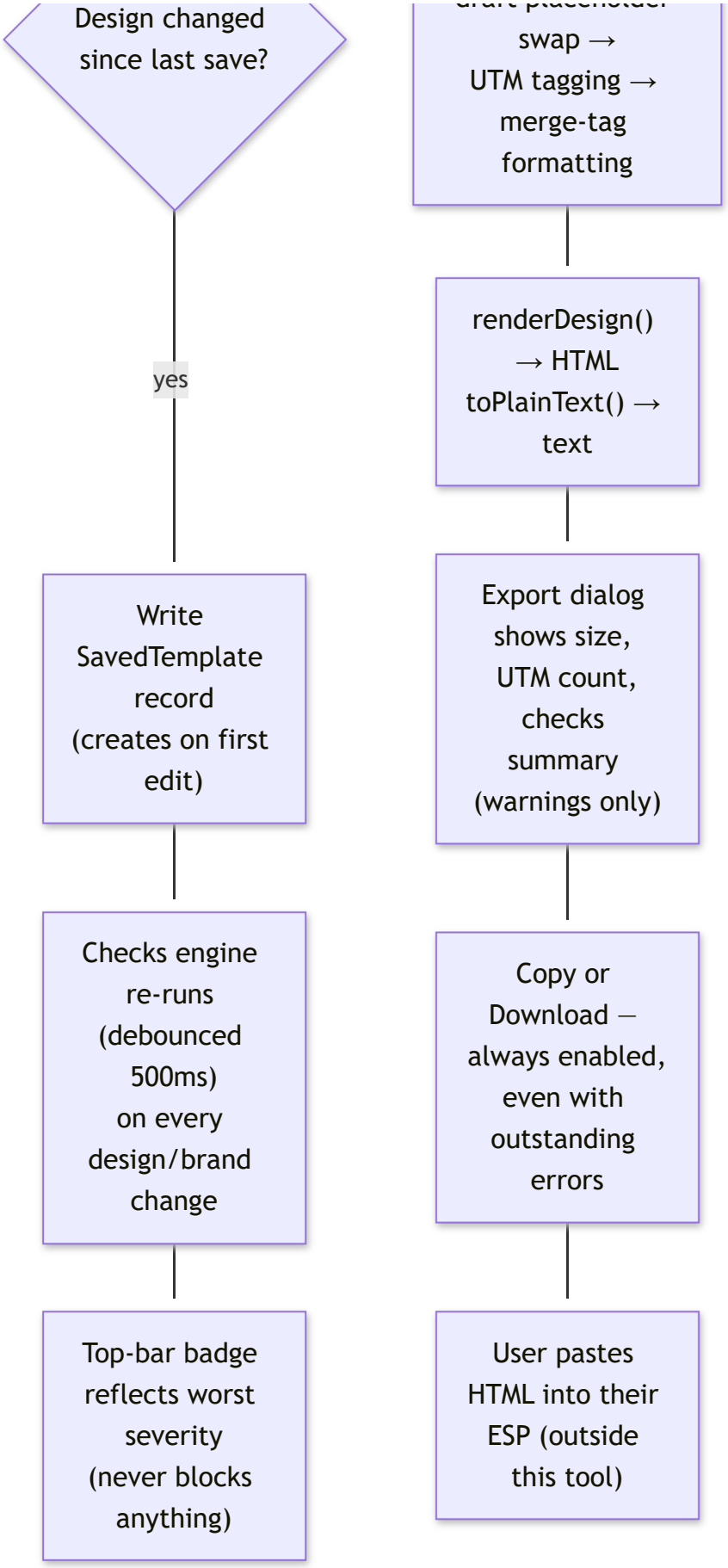
Function	Purpose	Business objective	Trigger	Inputs	Outputs	Dependencies	Users involved	Preconditions	Postconditions
Manage the image library	Centralize vetted image URLs	Consistency, fewer broken images	Images tab / any image field's picker	A public https URL	A `LibraryImage` record	`image-storage.ts`, `image-audit.ts`	Editor	URL passes https/malformed checks	Available via picker on every image field
Configure AI	Connect and route AI providers	Controlled AI spend/behavior, resilience	Settings → AI	API key, per-task model/provider/fallback choice	Stored key(s) (opt-in) + per-task routing config	`key-store.ts`, `tasks.ts`, `task-settings.ts`	Editor	—	Tasks (`text.brief`, `image.generate`) become "ready"
Back up / restore / clear local data	Data portability and recovery	Avoid total data loss; move between browsers	Settings → Data & storage	A backup JSON file, or the typed word "DELETE"	A downloaded JSON file / an overwritten local state / a wiped local state	`backup.ts`, `clear-all.ts`	Editor	For restore: a valid backup JSON; for clear-all: typed confirmation	Local state replaced or wiped; app reloads

7. Complete Business Process

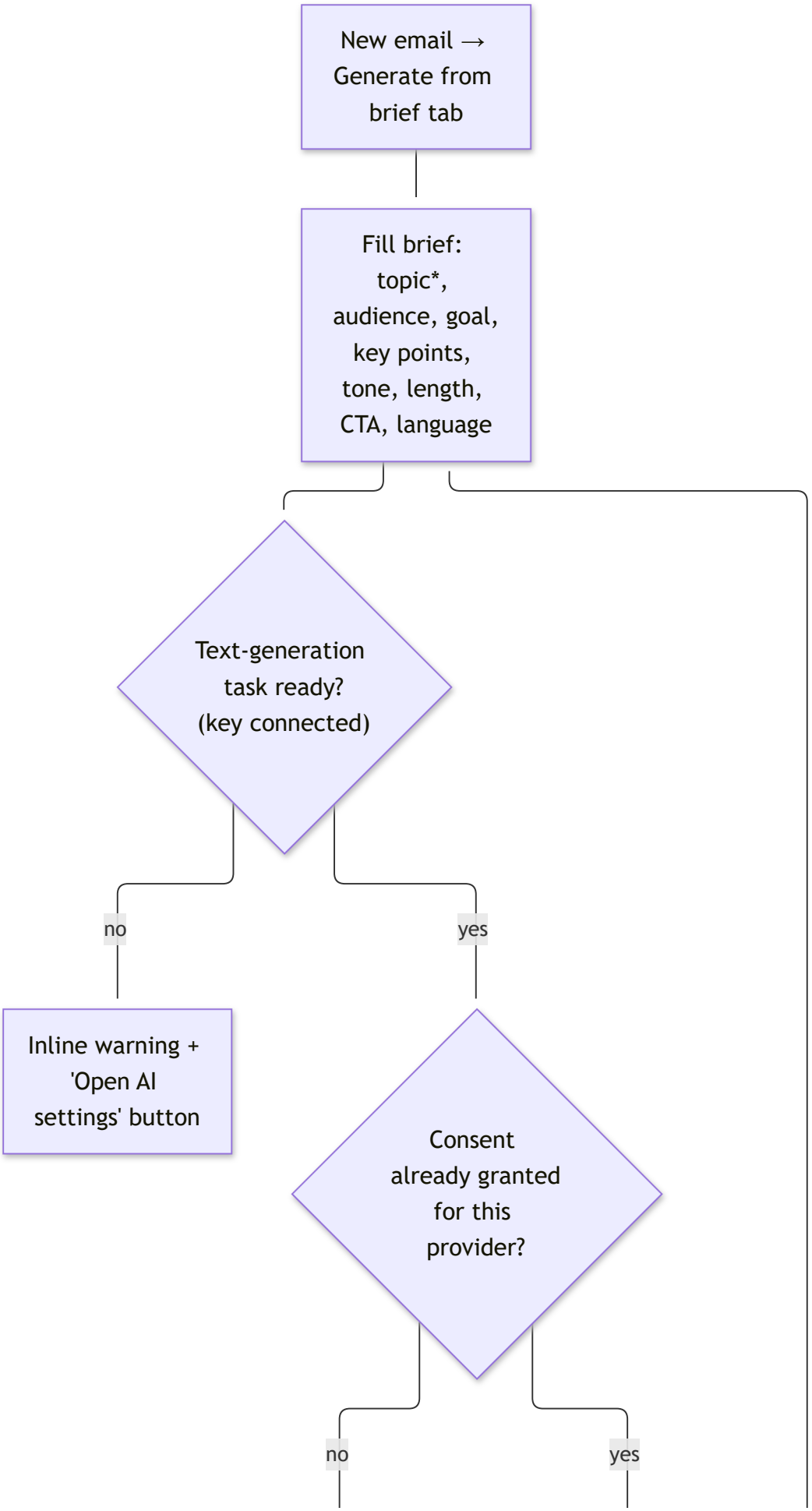
Because there is no multi-party approval chain, "business process" here means the **in-browser editorial workflow** — the closest analogue to a BPMN approval flow is the AI-brief human-review gate (6A) and the quality-check gate (3A), both of which are advisory rather than blocking.

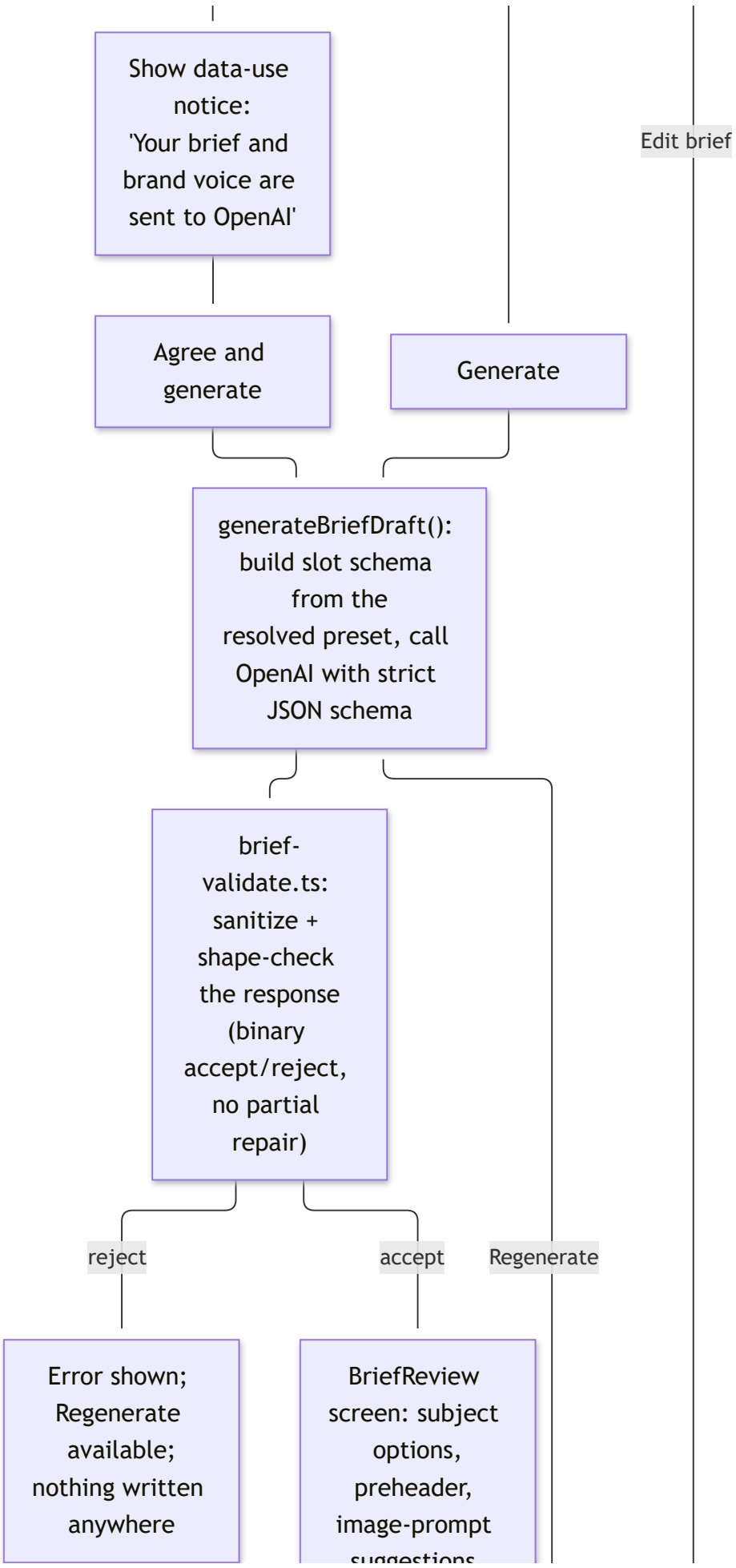
7.1 Primary process — compose and export an email (happy path)

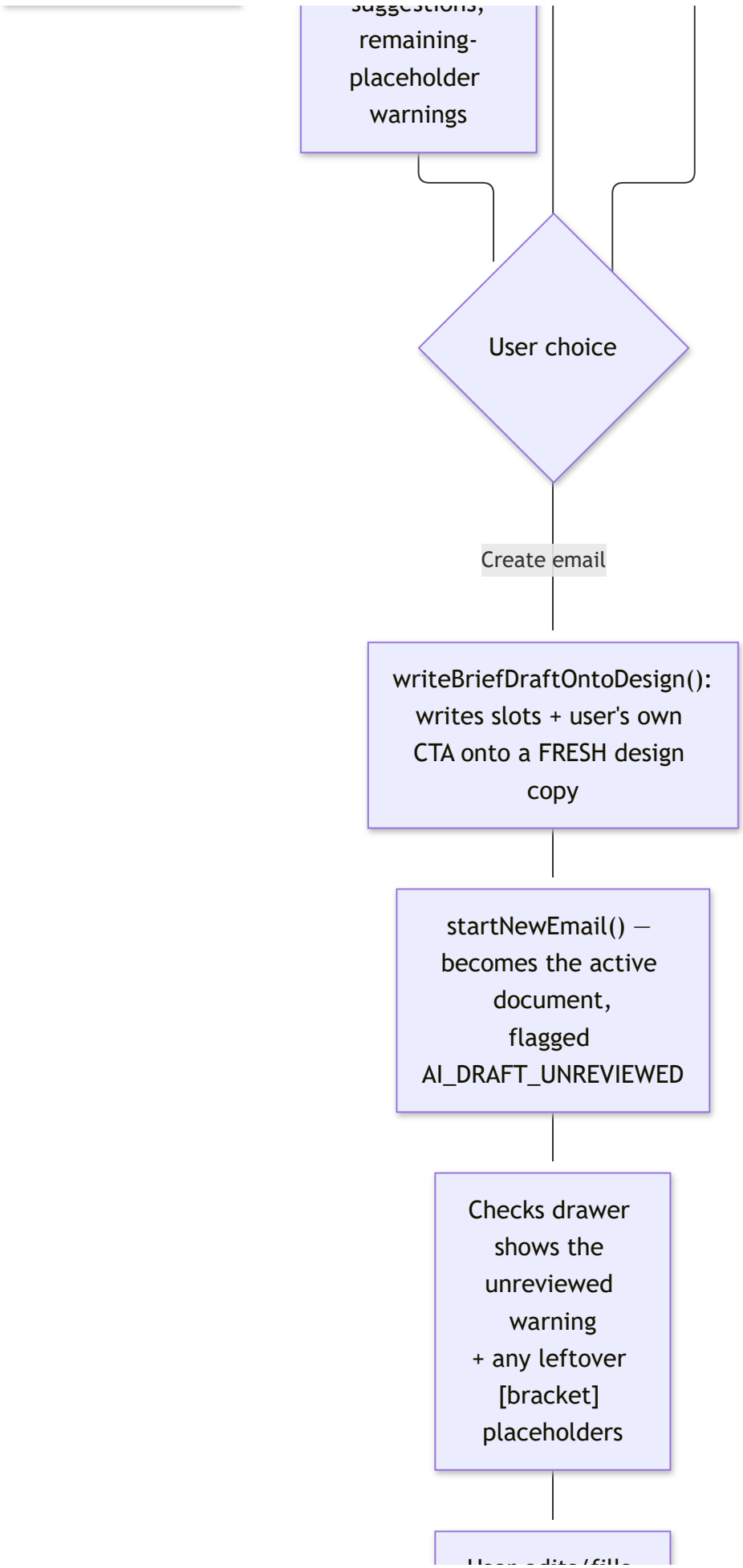

User opens app
First run?
yes
no
Seed: Vee brand + Welcome preset
Reopen last-edited email (email-builder:session:v1)
Canvas with active design
User edits: drag/add/configure blocks
Auto-save debounce (2s)
no
User opens Export
resolveForOutput(): brand tokens → AI-

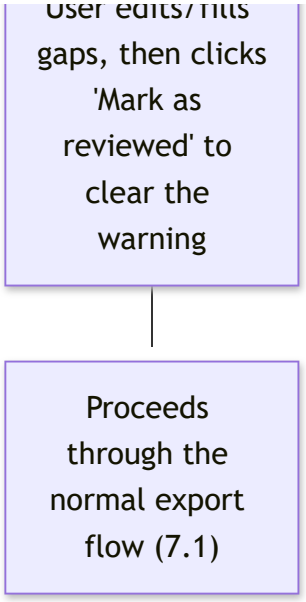

Design changed since last save?
yes
Write SavedTemplate record (creates on first edit)
Checks engine re-runs (debounced 500ms) on every design/brand change
Top-bar badge reflects worst severity (never blocks anything)
swap → UTM tagging → merge-tag formatting
renderDesign() → HTML toPlainText() → text
Export dialog shows size, UTM count, checks summary (warnings only)
Copy or Download — always enabled, even with outstanding errors
User pastes HTML into their ESP (outside this tool)

7.2 Alternate path — AI-assisted draft (6A), human review mandatory


New email → Generate from brief tab
Fill brief: topic*, audience, goal, key points, tone, length, CTA, language
Text-generation task ready? (key connected)
no
yes
Inline warning + 'Open AI settings' button
Consent already granted for this provider?
no
yes


Show data-use notice: 'Your brief and brand voice are sent to OpenAI'
Edit brief
Agree and generate
Generate
generateBriefDraft(): build slot schema from the resolved preset, call OpenAI with strict JSON schema
brief-validate.ts: sanitize + shape-check the response (binary accept/reject, no partial repair)
reject
accept
Regenerate
Error shown; Regenerate available; nothing written anywhere
BriefReview screen: subject options, preheader, image-prompt


suggestions,
remaining-placeholder warnings
User choice
Create email
writeBriefDraftOntoDesign(): writes slots + user's own CTA onto a FRESH design copy
startNewEmail() — becomes the active document, flagged AI_DRAFT_UNREVIEWED
Checks drawer shows the unreviewed warning + any leftover [bracket] placeholders



Business validation points: topic required and total brief length ≤4000 chars; CTA URL must be `https://` if supplied; a response exceeding any field's character budget is rejected outright (no partial acceptance).

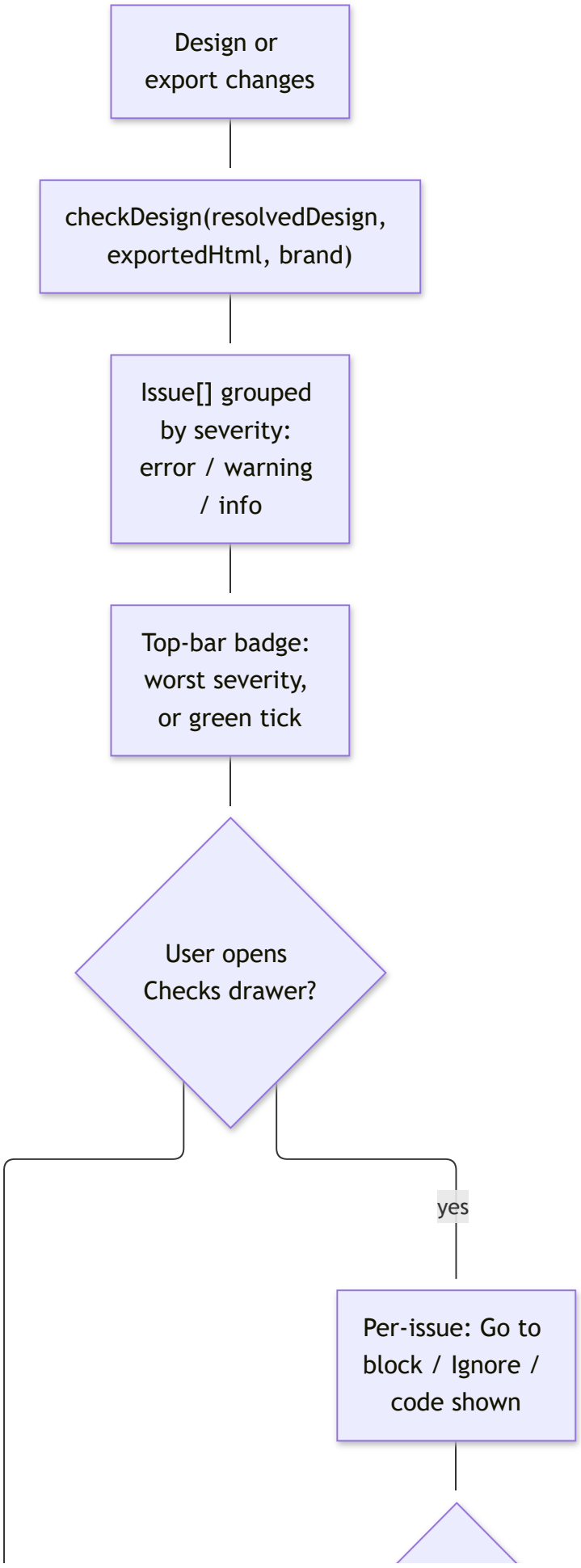
Decision points: Auto vs. explicit preset selection (goal→preset mapping, §13); consent granted vs. not; draft accepted vs. rejected by validation; human "Create" vs. "Regenerate" vs. "Edit brief."

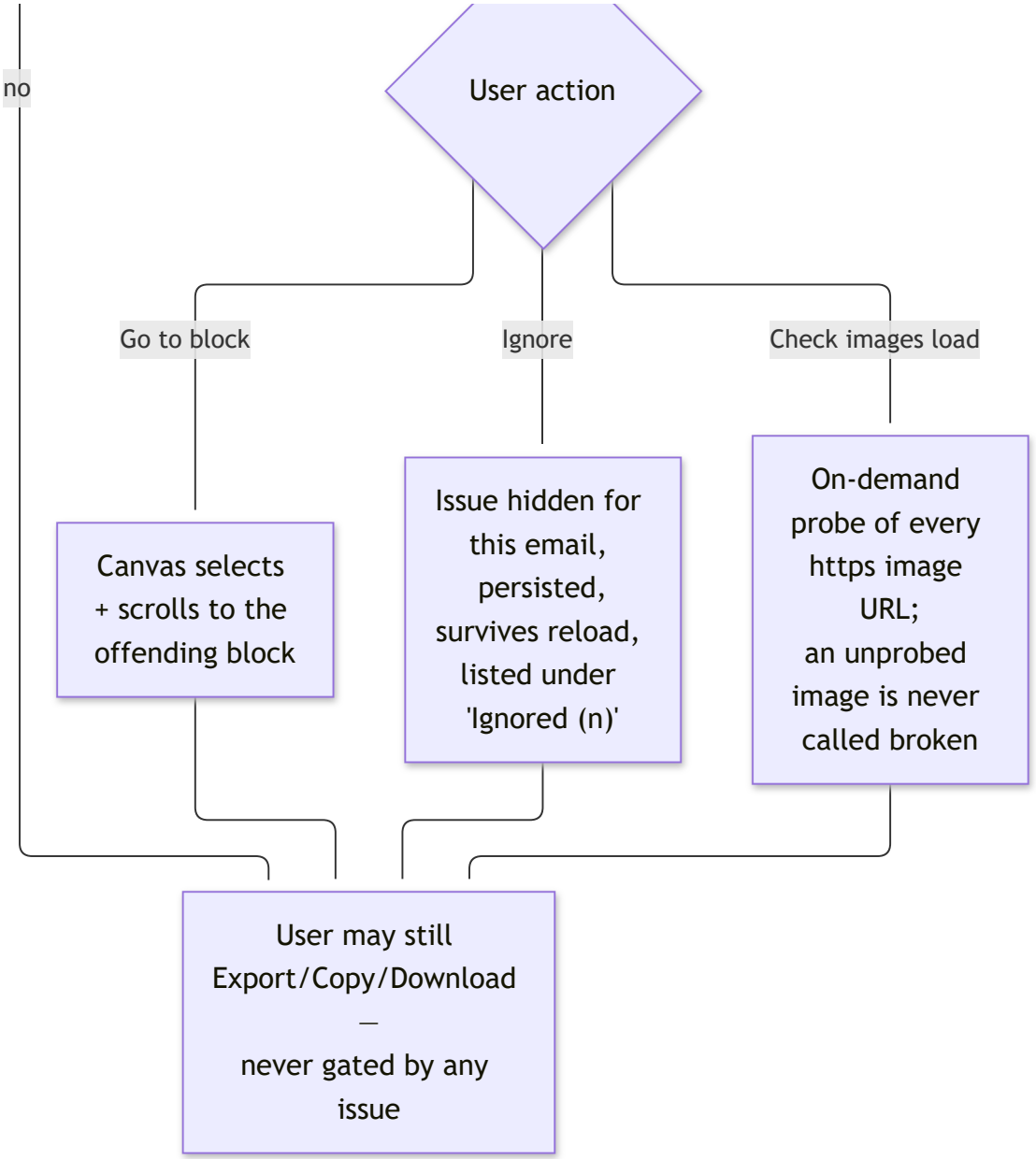
"Approval": the closest the product has to an approval step is the `AI_DRAFT_UNREVIEWED` check (a non-blocking warning) cleared by the same single user clicking **"Mark as reviewed."** There is no second-person sign-off anywhere in the codebase — **Not Found in Codebase.**

Failure/exception paths: provider/network failure → `AiError` surfaced with a redacted message (never the API key) and, for certain error classes, a configured fallback model is tried next (6B); a validation failure is always a full rejection with a "Regenerate" affordance, never a silent partial write.

Retry behavior — explicitly NOT automatic: both AI pipelines (image generation, brief generation) make **exactly one request per user action**; the product deliberately removed an earlier auto-retry because it "risked double-billing" — any further attempt requires an explicit user action (Regenerate) or a user-configured fallback chain (6B), never a silent same-request retry.

7.3 Alternate path — quality-check remediation (3A, non-blocking by design)


Design or export changes
checkDesign(resolvedDesign, exportedHtml, brand)
Issue[] grouped by severity: error / warning / info
Top-bar badge: worst severity, or green tick
User opens Checks drawer?
yes
Per-issue: Go to block / Ignore / code shown


User action
Go to block
Ignore
Check images load
Canvas selects + scrolls to the offending block
Issue hidden for this email, persisted, survives reload, listed under 'Ignored (n)'
On-demand probe of every https image URL; an unprobed image is never called broken
no
User may still Export/Copy/Download — never gated by any issue

8. Module Documentation

Module	Business purpose	Business owner	Users involved	Inputs	Outputs	Dependencies	Approval process	Notifications	Reports	Related modules
Canvas / Editor core	Visual composition of the email	Editor (sole role)	Editor	Block/row operations	Mutated `Design`, undo/redo history	`store.ts`, `renderer.ts`, `design-ops.ts`	None	Toast messages on actions	None	All others
Brand kit	Central visual identity (colors, fonts, logo, footer, social, voice)	Editor	Editor	Brand field edits	`Brand` record; resolved tokens at render time	`brand.ts`, `dark.ts`	None (`canEditBrand` always true)	None	Live kit preview in the Brands modal	Presets, Checks (dark contrast), AI brief (voice)
Presets	Ready-made starting structures	Editor (consumer only — presets are code, not user-edited)	Editor	Brand selection	A fresh `Design`	`presets.ts`, `defaults.ts`	None	Toast on apply	None	New Email modal, Brief generation (resolves onto a preset)

Module	Business purpose	Business owner	Users involved	Inputs	Outputs	Dependencies	Approval process	Notifications	Reports	Related modules
Brief generation (6A)	AI-assisted first-draft copy	Editor	Editor	Brief fields, brand voice	A reviewable draft; eventually a `Design` + `BriefRecord`	`create-from-brief.ts`, `slots.ts`, `brief-validate.ts`, OpenAI text API	Human "Mark as reviewed" (non-blocking)	Inline validation/error messages	None	Presets, Checks (`AI_DRAFT_UNREVIEWED`, `BRIEF_PLACEHOLDER_LEFT`)
AI image generation (5A)	On-brand imagery without leaving the editor	Editor	Editor	Prompt, aspect, style, quality	A local draft image (IndexedDB)	`openai.ts`, `validate.ts`, `drafts.ts`	None	Inline error cards with redacted diagnostics	None	Checks (`AI_IMAGE_MISSING_DRAFT` / `AI_IMAGE_NOT_HOSTED`), Image fields
AI provider/key management	Securely connect to OpenAI	Editor	Editor	API key(s)	Stored (opt-in) key record; connection status	`key-store.ts`	"Test and save" gate (key only stored once proven)	Status chip (Not configured / Configured / Connected)	None	AI image, Brief generation, Settings → AI
Per-task AI model routing (6B)	Independent model choice per capability, with fallback	Editor	Editor	Provider/model/options/fallback choices per task	A `TaskRoute` per task	`tasks.ts`, `task-settings.ts`, `run-task.ts`	None	Status chip; "No access to `<model>`" denial chip	None	AI image, Brief generation

Module	Business purpose	Business owner	Users involved	Inputs	Outputs	Dependencies	Approval process	Notifications	Reports	Related modules
Checks (quality engine, 3A)	Non-blocking in-box-safety review	Editor	Editor	Resolved design + exported HTML	`Issue[]`	`checks.ts`, `contrast.ts`, `image-audit.ts`	None (never blocks)	Top-bar badge; drawer	None (the drawer itself is the "report")	Export, Image library, Dark mode, Brief generation
Image library (2C-1)	Centralized vetted image URLs	Editor	Editor	Pasted https URLs	`LibraryImage[]`	`image-storage.ts`	None	Load-probe badges ("Could not load")	None	Checks, every image-bearing block field
Saved blocks library (2B)	Reusable content blocks/rows	Editor	Editor	A selected block/row + name/category	`SavedBlock[]`	`blocks-library.ts`	None	Toasts on save/import/overwrite	None	Canvas, Export/Import JSON
Auto-save / version history (2B)	Prevent data loss; enable "undo across sessions"	Editor	Editor	Every design change	`SavedTemplate` updates; `Version-Record` snapshots	`storage.ts`, `versions.ts`	None	Top-bar save status (incl. "storage full")	Storage usage read-out	Templates modal, History modal
Export / merge tags / UTM / plain text (2A)	Produce the shippable deliverable	Editor	Editor	Design, brand, merge/UTM settings	HTML + plain text	`output.ts`, `merge.ts`, `utm.ts`, `plaintext.ts`, `render.ts`	None	UTM tagged/skipped count; checks summary	None	Checks, Dark mode

Module	Business purpose	Business owner	Users involved	Inputs	Outputs	Dependencies	Approval process	Notifications	Reports	Related modules
Dark mode (3B)	Correct appearance in dark-mode-aware email clients	Editor	Editor	Brand dark palette (or derived)	Additive dark CSS in the export	`dark.ts`, `output.ts`	None	`DARK_NO_LOGO` / `DARK_IMAGE_MANUAL` / `SIZE_DARK_OVERHEAD` checks	None	Brand kit, Checks, Preview
Settings hub (7A)	Single home for configuration	Editor	Editor	—	—	`SettingsHub.tsx` + 7 section components	Danger-zone "type DELETE" gate	—	None (a shell over the modules above)	Profile, AI, Brands, Email defaults, Checks, Data & storage, About
Profile (7A)	Local "who is this" label	Editor	Editor	Display name	`createdBy` / `updatedBy` attribution	`user.ts`	None	One-time "Set your name →" hint	None	Brand kit, Templates
Data & storage / backup (7A)	Portability and recovery of local state	Editor	Editor	A backup JSON, or typed "DELETE"	A downloaded JSON / overwritten or wiped local state	`backup.ts`, `clear-all.ts`	Typed-confirmation gate on delete	Storage usage bar; import summary	Storage usage bar	All persisted modules except AI keys/drafts/consent (deliberately excluded)

9. Feature Documentation

Each feature below is drawn directly from `docs/PROGRESS.md`'s own acceptance narrative, cross-checked against source.

9.1 Feature 1A — Brand kit + brand switcher

- **Business Purpose:** Let the team re-theme any email instantly without touching its content.
- **Description:** A `Brand` entity (colors, fonts, button style, logo pairs, footer, social links) resolved into the design at render time.
- **Business Need:** Multiple campaigns/sub-brands must look consistent without manual recoloring per email.
- **Actors:** Editor.
- **Trigger:** Opening the Brands modal / switching the brand selector in the top bar or New-email dialog.
- **Preconditions:** None (a seed "Vee Technologies" brand exists on first run).
- **Workflow:** Create/edit a brand → fields stored as `Brand.tokens` → every color/font field in a design stores either a brand token (`"brand.primary"`) or a literal override → `resolveDesign(design, brand, mode)` replaces tokens with literals immediately before render/export.
- **Validation Rules:** Brand name required to save; logo upload ≤2MB, downscaled to 400px wide, types png/jpeg/svg+xml/webp.
- **Business Rules:** Switching brands changes only `design.brandId` — manual overrides survive; "Reset to brand" rewrites the 5 body-level tokens; token resolution order is active brand → seed brand → `''` ("inherit"); exports always use the hosted `logo_url`, never the uploaded preview logo (email clients strip `data:` URIs).
- **Exception Cases:** A `brandId` pointing at an archived/deleted brand silently falls back to the default brand.
- **Outputs:** A themed `Design`; a resolved, token-free export.
- **Notifications:** Toast on save/duplicate/import.
- **Dependencies:** `dark.ts` (dark palette derivation), every renderer that consumes color/font fields.
- **Related Features:** 1B (composite blocks consume brand footer/social/logo), 3B (dark palette), 6A (brand voice).

9.2 Feature 1B — Composite blocks (Header, Hero, Footer, Article list)

- **Business Purpose:** Stop newsletters being hand-assembled from atomic blocks for every common section.
- **Workflow:** Each composite block internally builds a throwaway sub-block (menu/social/button) and renders it through the real atomic renderer — no duplicated HTML logic.
- **Business Rules:** A block can never paint edge-to-edge (wrapped in a content-width container); Header/Footer have an "Extend background full width" switch that writes color onto the parent row instead (last-writer-wins if two such blocks share a row); Hero's centered background image uses `<td background>` (no VML — Outlook/Windows shows the fallback color only); `showFooter` (default

on) hides Footer blocks from preview/export only, while the canvas keeps them visible at reduced opacity with a "Hidden on send" badge; Hero card count tracks the chosen layout (2 for `cards-2`, 3 for `cards-3`), padding/trimming automatically on layout change.
- **Validation Rules:** Image `alt` text is required in the editor (red asterisk + warning) because inboxes block images by default.
- **Exception Cases:** Pre-1B saved designs are migrated with `showFooter: true` defaulted.
- **Related Features:** 1C (presets compose these four), 2C-1 (image fields on these blocks get the picker/audit).

9.3 Feature 1C — Starter presets

- **Business Purpose:** Give "New email" a useful starting point instead of a blank canvas.
- **Workflow:** "New email" → Presets tab → pick Blank or one of 8 presets (Newsletter, Promotional, Product launch, Welcome, Cold outreach, Event invitation, Order confirmation, Weekly digest) → a themed, fresh-id copy is built and becomes the active document.
- **Business Rules:** Every preset is brand-neutral (colors/fonts as tokens); the brand **name** is interpolated into prose once at apply time, then becomes ordinary editable text (a later brand switch re-themes colors but not that wording); every preset ships placeholder art/copy **on purpose**, specifically so the `IMAGE_PLACEHOLDER` / `LINK_PLACEHOLDER_DOMAIN` checks visibly flag it until replaced.
- **Outputs:** A live, real-HTML thumbnail per preset, themed with whichever brand is selected in the dialog.
- **Related Features:** 1B (composition source), 6A (presets are the structural target of brief generation), 3A (placeholder checks).

9.4 Feature 2A — Merge tags, UTM tagger, plain text

- **Business Purpose:** Make an exported email ESP-ready, trackable, and accessible without an HTML client.
- **Workflow:** Content always stores canonical `{{tag}}` merge syntax; at export, `resolveForOutput()` applies (in order) brand resolution → AI-draft placeholder swap → UTM tagging (export mode only) → ESP tag reformatting → render/plain-text.
- **Business Rules:** 6 built-in merge tags (`first_name`, `last_name`, `company`, `email`, `unsubscribe_url`, `view_in_browser_url`) map to real Mailchimp/HubSpot field syntax, not a naive transform; UTM tagging is off by default, applies at export only, never overwrites an existing `utm_*` param, and skips non-http(s) links, merge-tag-only URLs, the unsubscribe/view-in-browser links, and any link with its own opt-out flag; per-link opt-out (`noTrack`) is always stripped from the exported HTML, whether UTM is on or off; plain text wraps at 72 characters, never splitting a URL across lines.
- **Outputs:** Export dialog with HTML | Plain text tabs, each with Copy/Download; a UTM "tagged N, skipped M" summary.
- **Related Features:** Checks (`UNSUBSCRIBE_MISSING`, link rules operate on the same resolved/exported pair).

9.5 Feature 2B — Saved blocks, auto-save, version history

- **Business Purpose:** Remove the "lost work" and "rebuild it again" risks.

- **Workflow:** Every edit marks the email dirty and restarts a 2-second auto-save debounce, which creates the record on first edit; version snapshots are taken automatically (≤1 per 5 minutes of active editing), manually ("Save version"), and immediately before any restore or canvas replacement; the last 20 versions per email are kept, oldest pruned first; a "Save as block" action freezes a block/row payload into a searchable library, re-themeable per active brand (tokens kept as tokens).
- **Business Rules:** Restoring a version snapshots the present first, so a restore is itself always reversible; inserting a saved block always mints fresh ids (`cloneWithNewIds`), so the inserted copy can never alias the library original; storage write failures are never silent — the user sees "Not saved: storage full," and the app's quota-recovery strategy sacrifices old version snapshots first, never brand/template/image data.
- **Related Features:** Templates modal, History modal, Data & storage (backup/restore of the same records).

9.6 Feature 2C-1 — URL-based image library

- **Business Purpose:** Stop broken/inconsistent/un-vetted images from reaching an email.
- **Workflow:** Paste one or more public https URLs → each is validated (scheme, shape) and probed (does it actually load) → accepted entries get a name, default alt text, tags, and optional brand scope → a picker button appears on every one of 12 image-bearing fields app-wide, filling both the URL and its default alt text.
- **Business Rules:** File upload is deliberately refused ("Upload needs image hosting. Paste a public https image URL instead.") rather than silently falling back to an embedded `data:` URI, because most inboxes strip those; a structural problem (not-https, missing alt, placeholder pattern) is reported instantly, but a "broken" verdict requires an explicit "Check images load" probe — an unexamined image is never accused.
- **Related Features:** Checks (shares the same audit data), every block with an image field, Brand logo fields.

9.7 Feature 3A — Checks (quality engine)

- **Business Purpose:** Tell the sender, before they send, what an inbox will actually do with this email — without ever stopping them from sending anyway.
- **Workflow:** `checkDesign(resolvedDesign, exportedHtml, brand, opts)` runs on the exact pair the Export dialog itself builds, debounced 500ms, also re-run on a whole-design load.
- **Business Rules (full catalogue in §13):** size vs. Gmail's ~102KB clip; dead/insecure/placeholder-domain links; missing unsubscribe text; empty/over-long subject and preheader; WCAG AA contrast (text, buttons, links, light and dark); text under 12px; buttons under a 44px tap target; thin/image-only content; every image-quality rule; AI-draft-specific rules (missing/unhosted draft, unreviewed brief, leftover brief placeholders).
- **Exception Cases:** Text sitting on a hero background image is explicitly reported as "unknown," never guessed at.
- **Outputs:** A severity-grouped drawer with per-issue "Go to block"/"Ignore," and a top-bar badge.
- **Related Features:** Every content/export feature feeds into this engine; none of them can be blocked by it.

9.8 Feature 3B — Dark mode support

- **Business Purpose:** Correct appearance in dark-mode-aware clients (Apple Mail, Outlook Mac/iOS, Outlook.com) without breaking light mode.
- **Workflow:** The design is resolved twice — once against the real brand, once against a dark-palette-swapped brand — and the two renders are diffed: a value that moved came from a brand token (gets a CSS class); a value that stayed was a hand-typed hex (stays exactly as typed). A manual per-block/per-row override beats both.
- **Business Rules:** Fully additive — with the toggle off, the export is byte-identical to a pre-dark-mode export (no meta tags, no stylesheet, no class names); two hand-corrected role-mapping cases exist (an "ink" color used as a background must not flip to light text; a "surface" color used as a label must resolve to whatever actually reads on its real background) because brand kits name colors by identity, not role.
- **Exception Cases:** Gmail's own mail apps ignore all of this and apply their own inversion (explicitly noted in the Preview UI); a hero background image cannot be restyled for dark mode at all (`DARK_IMAGE_MANUAL` info).
- **Related Features:** Brand kit (dark palette fields), Checks (dark-specific contrast rules), Preview.

9.9 Feature 5A — AI image generation (bring-your-own key, with 4 amendments)

- **Business Purpose:** Generate on-brand images for an email without leaving the editor or paying for a separate tool/subscription.
- **Workflow:** Open the AI image panel from any of 5 image-bearing field types → describe the image, choose aspect/style/quality → one request to OpenAI's image endpoint → the result becomes a local "draft" (bytes in IndexedDB, only an id in the design) → "Use this image" requires alt text before it can be placed on canvas.
- **Business Rules:** Exactly one request per generation, never retried automatically (a failed-but-maybe-successful call "may already have been produced and billed"); the response is validated by magic bytes, not declared content-type, SVG is refused outright (can carry script), max 10MB; the API key defaults to in-memory-only (dies with the tab) and is never written into any serializable app state (swept by dedicated tests across every export/backup surface); because no image hosting exists, every export of a generated image ships a visible placeholder URL until the user manually hosts the PNG elsewhere and pastes the resulting URL in.
- **Exception Cases:** `AI_IMAGE_MISSING_DRAFT` (the browser doesn't hold the bytes — e.g., an imported design) vs. `AI_IMAGE_NOT_HOSTED` (the bytes exist locally but nowhere public) are two distinct, mutually exclusive checks, both reported only after the app has actually looked.
- **Related Features:** 6B (per-task model routing supersedes this feature's own single-provider model picker), Checks, Image fields.

9.10 Feature 6A — Generate an email from a brief

- **Business Purpose:** Produce a reviewable first-draft email from a short structured brief instead of a blank canvas or a hand-written draft.

- **Workflow:** Full BPMN-style flow in §7.2. Key mechanic: the set of AI-fillable fields ("slots") is **derived by walking the actual block tree** of the resolved preset and classifying by block type — not hand-written per preset — so all 8 presets are supported automatically and a 9th preset would be too.
- **Business Rules:** A field is AI-required exactly when the preset's own placeholder copy for it is non-empty (the preset author's choice to fill it in is itself the signal that it matters); AI is categorically excluded from footer/legal copy, structural blocks, logos, and the hero background image; if the user supplies their own CTA label, the AI is never even asked for one; the model's output is validated by a strict JSON Schema (`additionalProperties: false`, every field explicitly required-or-nullable) as a mechanical defense against prompt injection, on top of explicit delimiter-wrapped "this is data, not instructions" framing in the system prompt; validation is binary (accept the whole response or reject it with a reason) — no partial repair.
- **Exception Cases:** Bracket placeholders like `[add date]` are deliberately left alone by the sanitizer — they are the intended signal to the human that a fact was missing from the brief.
- **Outputs:** A `BriefDraft` (subject options, preheader, slot values, image-prompt suggestions) reviewed in `BriefReview` before anything is written to any design.
- **Related Features:** Presets (structural target), Checks (`AI_DRAFT_UNREVIEWED`, `BRIEF_PLACEHOLDER_LEFT`), 6B (model routing), Brand kit (voice guidance consumed here).

9.11 Feature 6B — Per-task AI model routing (+ amendment)

- **Business Purpose:** Let text generation and image generation use independently chosen models, with resilience via a configurable fallback chain, instead of one global "active provider."
- **Workflow:** Settings → AI → "Models per task" — one row per task (`text.brief`, `image.generate`), each with its own provider/model pick (or a free-typed model id), its own quality/temperature option, its own "Test connection," and its own ordered fallback list.
- **Business Rules:** A fallback candidate is tried only on a `network`/`timeout`/`rate-limit`-classified failure — never on a bad key, no-credit, content-policy refusal, or malformed response, because "a different model on the same account wouldn't fix any of those"; a candidate with no configured key is skipped; a per-generation Quality selector in the chat UI always wins over the task's configured default quality (a regression caught before shipping); defaults were reverted in the 6B amendment to "a model the user actually chose" (`gpt-4.1-mini` / `gpt-image-2`) rather than whatever is newest, though newer ids remain selectable.
- **Exception Cases:** A session-only, per-task "this model was just refused" flag (`model-access.ts`) swaps the status chip to "No access to `<model>`" and persists the warning until the model is changed and changed back.
- **Related Features:** 5A, 6A (both now call `runTask()` rather than hardcoding a provider/model), Settings hub.

9.12 Feature 7A — Profile bar and Settings hub

- **Business Purpose:** Consolidate previously scattered configuration (brand switcher only in the top bar, AI keys buried in a side-panel gear icon, no storage visibility, no backup) into one place, and give the product a local "who is this" identity substitute for an account.

- **Workflow:** A left-panel "Settings" gear opens a two-level hub (section list → section detail, Escape navigates back one level at a time) with 7 sections: Profile, AI, Brands, Email defaults, Checks, Data & storage, About.
- **Business Rules:** "Email defaults" apply **only** to a brand-new email at the moment of creation — never to an already-open or already-saved email, by construction (one function called from exactly 3 design-creation sites); "Clear all local data" requires literally typing the word `DELETE` before the button enables, and clears AI keys and IndexedDB drafts along with everything else; "Export all data" (backup) structurally cannot reach the AI key store or drafts because its own file never imports those modules.
- **Related Features:** Every module above gets a home here; Brands (management), AI (keys + routing), Data & storage (backup/restore of everything else).

---

10. Screen Documentation

All navigation is **client-side React state** — there is **no URL router** (confirmed: `App.tsx` uses a `ModalName` union plus component state, not `react-router` or any history-API usage). Every "screen" below is therefore a **modal, side panel, or in-place view**, not a route, and there are no deep-linkable URLs. (Fact.)

Screen / Component	Type	Business purpose	Opened from	Key fields	Buttons / actions	Validation shown	Navigates to
Editor shell (`App.tsx`)	Root layout	Hosts every other screen	App load	—	Keyboard shortcuts (Ctrl+Z/Shift+Z/Y, Ctrl+D, Delete, Escape)	—	Everything below
Top bar (`TopBar.tsx`)	Persistent header	Title, brand switch, undo/redo, device/scheme toggle, save status, all modal launchers	Always visible	Email title, brand selector	New, Templates, History, Import JSON, Download JSON, Export, Checks badge, Preview	Confirms replace if `dirty` before New/Import	New Email, Templates, History, Export, Preview modals
Left panel (`LeftPanel.tsx`)	Persistent sidebar	Add content; manage saved blocks/images; entry to Settings	Always visible	4 tabs: Content, Rows, Saved, Images	Tile-click-to-add; drag-and-drop; Settings gear; Profile row	Saved-block import reports duplicates/clashes	Settings hub, Image picker
Canvas (`Canvas.tsx`)	Main view	WYSIWYG preview = the actual export renderer	Always visible	—	Drag/reorder/select/duplicate/delete	Dimmed "Hidden on send" badge for hidden footers	Right panel (via selection)

Screen / Component	Type	Business purpose	Opened from	Key fields	Buttons / actions	Validation shown	Navigates to
Right panel (`RightPanel.tsx`)	Persistent sidebar	Property editor for the current selection, or email-wide settings when nothing is selected	Always visible (replaced by AI image panel when generating an image)	Varies per block type	Save as block, Duplicate, Delete (block/row); Reset to brand (body)	Per-field (e.g., CTA URL https-only)	AI image panel, Save Block dialog
New email modal (`NewEmailModal.tsx`)	Modal	Start a new email from a preset or a brief	Top bar "New"	Brand selector; preset grid or brief form	Pick a card; "Generate"/"Create email"	Confirms replace if `dirty`	Brief form/review (embedded), closes into Canvas
Brief form (`BriefForm.tsx`)	Embedded view (inside New Email modal)	Collect brief inputs	"Generate from brief" tab	Topic*, audience, goal, key points (≤8), tone, length, CTA label/URL, article count, language, preset, source text	Add/remove key point; "Agree and generate" / "Generate"	Topic required; total ≤4000 chars; CTA URL https-only; "needs an OpenAI key" warning if not ready	Brief review
Brief review (`BriefReview.tsx`)	Embedded view	Human review/approval of the AI draft before creation	After a successful generation	Subject options + editable subject (≤ `SUBJECT_MAX_CHARS`), preheader (read-only), image-prompt suggestions	Create email; Regenerate; Edit brief; Copy (per image prompt)	Lists every leftover `[bracket placeholder]` before Create	Canvas (on Create)

Screen / Component	Type	Business purpose	Opened from	Key fields	Buttons / actions	Validation shown	Navigates to
Brands modal (`BrandsModal.tsx`)	Modal	Full brand-kit CRUD	Top bar palette icon / Settings → Brands → "Manage brands"	Identity, Logos, Colors, Dark mode, Fonts, Buttons, Footer, Social links, Voice	New brand, Import JSON, per-brand Export/Duplicate/Edit/Archive/Restore	Name required to save; dark-palette readable-contrast readout	Brand edit form (embedded)
Templates modal (`Modals.tsx::TemplatesModal`)	Modal	Browse/open/rename/delete saved emails	Top bar "Templates"	Email name field	Rename this email; Save as a copy; Open; Delete	Confirms replace if `dirty`; storage usage bar with low-space warning	Canvas (on Open)
History modal (`HistoryModal.tsx`)	Modal	Version history browse/restore	Top bar "History"	Optional note field	Save version; Preview; Restore	"No changes since the last version" when nothing changed	Read-only version preview (embedded)
Export modal (`Modals.tsx::ExportHtmlModal`)	Modal	Produce the final HTML/plain-text deliverable	Top bar "Export"	—	Copy, Download (.html/.txt), "Open checks →"	Checks summary line; UTM tagged/skipped count; size in KB	Checks drawer
Preview modal (`Modals.tsx::PreviewModal`)	Modal	See the exact exported HTML rendered	Top bar "Preview"	—	Desktop/Mobile toggle, Light/Dark toggle (if dark mode on)	Notes when showing local AI drafts vs. their export placeholder	—
Checks drawer (`ChecksPanel.tsx`)	Overlay (not a modal — layers over the right panel)	Review/act on quality issues	Checks badge (top bar) or "Open checks →" (export)	—	Go to block; Ignore/Unignore; "Check images load"	Groups by severity; "Ignored (n)" section	Canvas (block selection)

Screen / Component	Type	Business purpose	Opened from	Key fields	Buttons / actions	Validation shown	Navigates to
AI image panel (`AiImagePanel.tsx`)	Replaces right panel	Generate/use an AI image for the selected field	Any image field's "Generate with AI" entry point	Prompt, aspect, style, quality, brand-colors toggle	Send/Generate, Cancel, Use this image, Regenerate, Edit prompt, Download PNG	Alt text required before "Add to email"; readiness/settings warning if no key	Settings view (embedded, via gear icon)
AI settings view (`AiImagePanel.tsx::SettingsView`)	Embedded view (shared by the AI image panel's gear AND Settings → AI)	Manage provider API key(s)	AI panel gear / Settings → AI	API key (masked), Remember-on-device checkbox	Add/Replace key, Test and save, Test connection, Remove key	"Key is only stored once Test connection passes"	Back to chat / back to Settings
Image library (`ImageLibrary.tsx`)	Embedded view (Left panel "Images" tab) + **Image picker modal** when invoked from a field	Manage/choose vetted image URLs	Left panel tab / any image field's picker button	Search, brand filter (This brand/Shared/All)	Add, Export, Import, per-image Copy URL/Delete	Add flow validates + probes every pasted URL before accepting	Add-images flow (embedded)
Save Block dialog (`SaveBlockDialog.tsx`)	Modal	Save a selected block/row to the reusable library	"Save as block" (right panel header)	Name* (required), Category (free text, autocompletes known categories)	Save block / Cancel	Warns if the payload embeds a `data:` image	—
Settings hub (`SettingsHub.tsx`)	Replaces left-panel content	Single home for configuration	Left panel Settings gear / Profile row	—	7 section rows	—	The 7 sections below
Settings → Profile (`ProfileSection.tsx`)	Embedded view	Set the local display name	Settings hub	Display name (≤40 chars)	Save	"A display name is required" if empty	—

Screen / Component	Type	Business purpose	Opened from	Key fields	Buttons / actions	Validation shown	Navigates to
Settings → AI (`AiSection.tsx`)	Embedded view	Keys + per-task model routing	Settings hub	Per-task provider/model/option/fallback pickers	Test connection (per task); Reset to defaults; Add/remove fallback	Status chip per task; "No access to `<model>`" denial chip	Opens the shared AI settings view above it
Settings → Brands (`BrandsSection.tsx`)	Embedded view	Default brand + entry to full brand management	Settings hub	Default brand for new emails (drop-down)	Manage brands	—	Brands modal
Settings → Email defaults (`EmailDefaultsSection.tsx`)	Embedded view	Org-wide defaults for brand-new emails	Settings hub	Merge format (+ custom prefix/suffix), UTM source/medium, Dark mode toggle	— (auto-saves on change)	Explicit note: "never edits an email you already have open or saved"	—
Settings → Checks (`ChecksSection.tsx`)	Embedded view	Bulk-reset dismissed warnings	Settings hub	—	Reset for this email (n); Reset for every email	Confirmation dialog before reset	—
Settings → Data & storage (`DataSection.tsx`)	Embedded view	Backup/restore/clear local data	Settings hub	Delete-confirmation text field	Export all data; Import backup; Clear all local data	Import shows a record-count summary before overwrite; delete requires typing `DELETE`	Reloads the app after import/clear
Settings → About (`AboutSection.tsx`)	Embedded view	Version + keyboard shortcuts reference	Settings hub	—	Link to project README	—	—

Screen / Component	Type	Business purpose	Opened from	Key fields	Buttons / actions	Validation shown	Navigates to
Confirm dialog (`ConfirmDialog.tsx`)	Global modal (context-provided)	Reusable yes/no confirmation for destructive actions	Any destructive action app-wide	—	Confirm / Cancel	Danger styling when the action is destructive	Returns to caller

---

Part 2 of 4 — Sections 11–20

---

11. Navigation Flow

There is **no router and no menus/submenus in the traditional sense** — the whole app is one screen with a persistent top bar, left panel, canvas, and right panel, plus modals/overlays layered on top. (Fact — confirmed via `App.tsx`'s `ModalName` union and component state; no `react-router`, no `<a href>` navigation, no browser history entries created by in-app navigation.)

Role-based navigation: Not Found in Codebase — every screen is reachable by the single Editor role; there is no conditional menu rendering based on identity.

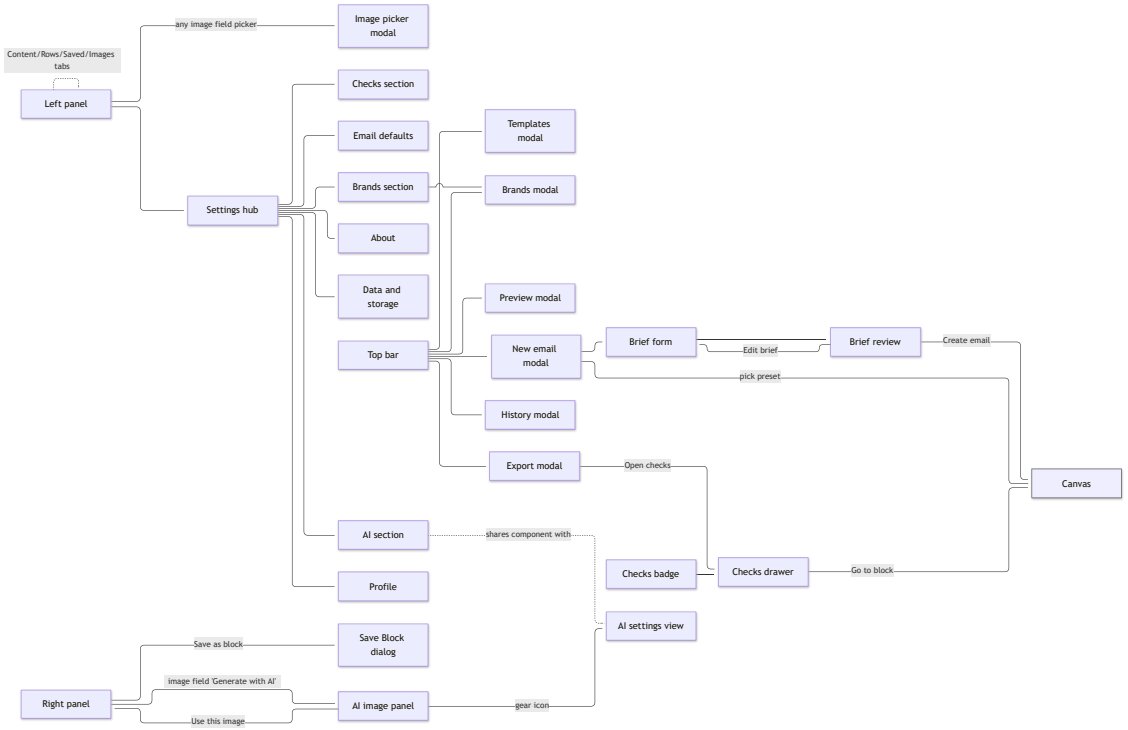
Conditional navigation that does exist (Fact):

- The AI image panel **replaces** the right panel entirely while open (not a modal) — selecting any other block/row underneath is still possible but the properties panel is hidden until "Back."
- The Checks drawer **overlays** (not replaces) the right panel, specifically so a block jumped to from the drawer has its property editor waiting underneath once the drawer closes.
- The Settings hub **replaces** the left panel's tab content; opening it snapshots the previously active tab + scroll position in a ref and restores both on close.
- On narrow viewports (`max-md:`), the Settings hub becomes a full-screen sheet (`fixed inset-0`) — the **only** responsive behavior anywhere in the app's own chrome (Fact, 7A).
- Several cross-component "jumps" exist without any prop-drilling, via a shared module-level callback registry pattern: `registerAiPanel`/`registerImagePicker` (5A, in `fields.tsx`) and `registerSettingsOpener` (6B, in `settings-nav.ts`) — e.g., the Brief form's "Open AI settings" button closes the New Email modal first (since it covers the whole screen) and then opens Settings → AI directly.

Hidden pages: Not Found in Codebase — nothing is hidden-but-reachable by URL; there is no URL surface at all.

Redirects: Not Found in Codebase — there is no routing layer to redirect within.

11.1 Navigation diagram (edges = "X opens Y")


Content/Rows/Saved/Images tabs
Left panel
any image field picker
Image picker modal
Settings hub
Checks section
Email defaults
Brands section
About
Data and storage
Templates modal
Brands modal
Preview modal
Top bar
New email modal
Brief form
Edit brief
Brief review
Create email
pick preset
History modal
Export modal
Open checks
Canvas
AI section
shares component with
Checks badge
Checks drawer
Go to block
Profile
AI settings view
Save as block
Save Block dialog
Right panel
image field 'Generate with AI'
Use this image
AI image panel
gear icon

12. User Journey

There is one role, so journeys are scoped by **task**, not by persona. Each is the full, runnable path through the screens in §10.

12.1 Journey — first-time setup and first email

```
App load (no prior data)
  ↓
Seed data created: default "Vee Technologies" brand + a Welcome preset design
  ↓
Canvas shown with the seeded design
  ↓
User edits text/images/blocks
  ↓
Auto-save creates the first SavedTemplate record (2s after first edit)
  ↓
(Optional) User opens Settings → Profile, sets a display name
  ↓
User opens Export, copies/downloads the HTML
  ↓
Completion: HTML pasted into the team's ESP (outside this tool)
```

12.2 Journey — AI-assisted email from a brief

```
Top bar "New" → New Email modal → "Generate from brief" tab
  ↓
Fill brief fields (topic required)
  ↓
Business Validation: total brief length ≤4000 chars, CTA URL https-only
  ↓
Decision Point: text-generation task ready? → if not, "Open AI settings" and configure a key
first
  ↓
Decision Point: consent already granted? → if not, data-use notice shown, "Agree and
generate"
  ↓
Generate → model call → brief-validate.ts shape/sanitization check
  ↓
Business Rule: binary accept/reject, no partial repair
  ↓
Brief Review screen: choose/edit subject, read preheader, copy image prompts
  ↓
Approval-equivalent: "Create email" (or Regenerate, or Edit brief)
  ↓
Data Update: writeBriefDraftOntoDesign() onto a fresh Design; email flagged
AI_DRAFT_UNREVIEWED
  ↓
Notification: Checks drawer shows the unreviewed warning + any leftover [bracket]
placeholders
  ↓
User fills gaps, clicks "Mark as reviewed"
  ↓
Completion: proceeds through the normal export journey (12.1, from "User opens Export")
```

12.3 Journey — generate and use an AI image

```
Select an image-bearing field → "Generate with AI"
  ↓
AI image panel replaces the right panel
  ↓
Decision Point: image-generation task ready? → if not, inline warning + "Open AI settings"
  ↓
Type a prompt, choose aspect/style/quality, optionally toggle brand colors
  ↓
Send (Enter, or the send button) → one request, no auto-retry
  ↓
Business Rule: result becomes a local "draft" — bytes in IndexedDB, only an id in the design
  ↓
User enters alt text (required) → "Add to email"
  ↓
Data Update: the field's aiDraftId is set; refreshDrafts() updates Checks' "known drafts" set
  ↓
Notification (Checks): AI_IMAGE_NOT_HOSTED warning — the export will ship a placeholder until
hosted
  ↓
Completion: user downloads the PNG and pastes a real hosted URL into the field later,
clearing aiDraftId
```

12.4 Journey — brand management and re-theme

```
Top bar palette icon (or Settings → Brands → "Manage brands") → Brands modal
  ↓
New brand / Edit existing → Identity, Logos, Colors, Dark mode, Fonts, Buttons, Footer,
Social, Voice
  ↓
Business Validation: name required to save
  ↓
Save brand → upsertBrand() stamps createdBy/updatedBy from the current Profile name
  ↓
Switch the active brand (top bar selector, or New Email dialog selector)
  ↓
Data Update: only design.brandId changes — content and any manual hex overrides are untouched
  ↓
Completion: every brand-token-sourced color/font across the open email re-themes instantly
```

12.5 Journey — recover from a mistake (undo, version history)

```
In-session mistake → Ctrl+Z (undo) / Ctrl+Shift+Z or Ctrl+Y (redo)
  ↓
(For an older mistake) Top bar "History" → History modal
  ↓
Select a version → Preview (read-only) → Restore this version
  ↓
Business Rule: the current design is snapshotted first, so the restore itself is reversible
  ↓
Completion: canvas replaced with the restored design; Checks re-run automatically
```

12.6 Journey — data portability (backup, restore, clear)

```
Settings → Data & storage
  ↓
"Export all data" → downloads one JSON file (brands, templates, saved blocks, images,
versions)
  ↓
(On a different browser/machine) "Import backup" → file parsed and shape-validated
  ↓
Notification: a record-count summary shown before anything is overwritten
  ↓
Decision Point: user confirms "Overwrite local data with this backup?"
  ↓
Data Update: importBackup() overwrites the five covered categories; AI key/drafts untouched
  ↓
Completion: app reloads
```

Alternate ending — **destructive path**:

```
Settings → Data & storage → "Danger zone"
  ↓
Business Validation: must type the literal word DELETE before the button enables
  ↓
Confirm dialog: "Clear all local data?"
  ↓
Data Update: clearAllLocalData() wipes AI keys, every IndexedDB draft, and every email-
builder:* localStorage key
  ↓
Completion: app reloads to a blank first-run state
```

---

13. Business Rules

Rules are grouped by the module they govern. Source is tagged per rule; all are **Fact** unless marked otherwise. Priority is **Inference** (Critical = would break core trust/safety guarantees if violated; High = a named, deliberately-engineered behavior; Medium = a UX/consistency rule).

Rule ID	Description	Condition	Action	Affected Modules	Priority	Source
BR-01	Resolve at the boundary	A color/font field holds a brand token or a literal	Tokens resolve to literals only once, immediately before render/export; the renderer never sees a token	Brand kit, Canvas, Export, Dark mode	Critical	`brand.ts`, `output.ts`
BR-02	Brand switch preserves overrides	User changes `design.brandId`	Only `brandId` changes; literal (hex) overrides and all content are untouched	Brand kit	Critical	`store.ts: setBrandId`
BR-03	Archived/missing brand fallback	`design.brandId` points at an archived or deleted brand	Silently resolves to the default brand	Brand kit	High	`brand.ts: resolveBrand`
BR-04	Export uses the hosted logo only	A brand has both an uploaded preview logo and a hosted `logo_url`	Export always uses `logo_url`/`logo_dark_url`; the upload is canvas-preview-only	Brand kit, Export	Critical	`brand.ts`
BR-05	Never block, only warn	Any Checks issue, image-audit issue, or placeholder-art warning exists	Copy/Download/Export remain enabled regardless of severity	Checks, Export	Critical	`checks.ts`, `Modals.tsx`

Rule ID	Description	Condition	Action	Affected Modules	Priority	Source
BR-06	No retry that could double-bill	An AI (image or text) generation request fails	Exactly one request is made; no automatic retry; a fallback chain (6B) is a distinct, user-configured subsequent call, never a same-request retry	AI image, AI brief, Model routing	Critical	5A amendment 4, 6B
BR-07	Never accuse before looking	An image URL or an AI draft reference has not yet been actively checked	It is never reported as broken/missing; only a probe (`Check images load`) or a `refreshDrafts()` read can produce that verdict	Checks, Image library, AI image	Critical	`image-storage.ts`, `store.ts: refreshDrafts`
BR-08	API keys never enter serializable state	Any export, backup, saved block, or design JSON is produced	The AI key is structurally absent (deliberately kept out of the zustand store) — swept by dedicated tests across every such surface	AI key management, Backup	Critical	`key-store.ts`, 5A/6A/6B tests
BR-09	Per-browser, export/import is the sharing mechanism	Brands, templates, saved blocks, images	No sync between browsers/devices exists; JSON export/import is the only transfer path	All persisted modules	Critical	README "Known limitations"

Rule ID	Description	Condition	Action	Affected Modules	Priority	Source
BR-10	No login, no roles	Any access-control decision	Every screen/action is available to whoever has the browser open; `createdBy` / `updatedBy` / `canEditBrand` / `getAccessibleBrands` are explicit future seams that currently return "everything"/" true"	Entire app	Critical	`user.ts`, `brand.ts`
BR-11	Auto-save on edit, not on load	Any design mutation vs. a whole-design load (open/restore /undo)	Only a mutation marks the email dirty and schedules a save; a load (detected by a `revision` bump) never does	Auto-save	High	`App.tsx`, `store.ts`
BR-12	Version dedup	A snapshot would be identical (deep-equal) to the latest stored version for that email	No snapshot is taken, regardless of the requested reason	Version history	Medium	`versions.ts: shouldSnapshot`
BR-13	Version cap and quota sacrifice order	Local storage nears/hits its quota	Version history (10 globally-oldest snapshots) is pruned first and automatically retried; brand/template/image/block data is never silently dropped	Storage	High	`storage.ts: writeSafe`

Rule ID	Description	Condition	Action	Affected Modules	Priority	Source
BR-14	Email-defaults scope	A setting is changed in Settings → Email defaults	Applies only to a brand-new email at the moment of creation (3 call sites); never mutates an already-open or already-saved email	Settings, New Email	High	`email-defaults.ts`
BR-15	Backup excludes secrets by construction	`exportBackup()` is called	The function's own file never imports the key-store or drafts modules, so no code path there can reach either	Backup	Critical	`backup.ts`
BR-16	Destructive-action confirmation	"Clear all local data"	Disabled until the user types the literal word `DELETE`; then requires a second confirm dialog	Data & storage	High	`DataSection.tsx`
BR-17	Image upload is categorically refused	A user attempts "Upload from device" on a content image field	The action throws/refuses rather than embedding a `data:` URI, because most inboxes strip those	Image library	Critical	`image-storage.ts`: `UploadUnavailableError`
BR-18	10-item repeater cap	An Article-list block	Items are capped at 10 everywhere they are consumed (collectors, renderers, text export)	Article list block	Medium	multiple files, consolidated

Rule ID	Description	Condition	Action	Affected Modules	Priority	Source
BR-19	UTM never double-tags	A link already carries any `utm_*` query parameter	UTM tagging leaves it untouched	Export	Medium	`utm.ts`
BR-20	UTM/tracking never touches the unsubscribe/ view-in-browser link	A link's URL is (or resolves to) `{{unsubscribe_url}}` / `{{view_in_browser_url}}`	Skipped from tagging but still **counted** in the "skipped" total so the reconciliation numbers are honest	Export	Medium	`utm.ts`
BR-21	Merge-tag canonical storage	Any text containing a merge tag	Always stored as `{{tag}}`; the output spelling (curly/Mailchimp/HubSpot/ custom) is chosen at Email settings and applied only at export	Export	High	`merge.ts`
BR-22	Dark-mode additive-only guarantee	`body.darkMode` is off	The export is byte-identical to a pre-dark-mode export — no meta tags, no stylesheet, no class names	Dark mode	Critical	`dark.ts`, `output.ts: darkRenderOpts`

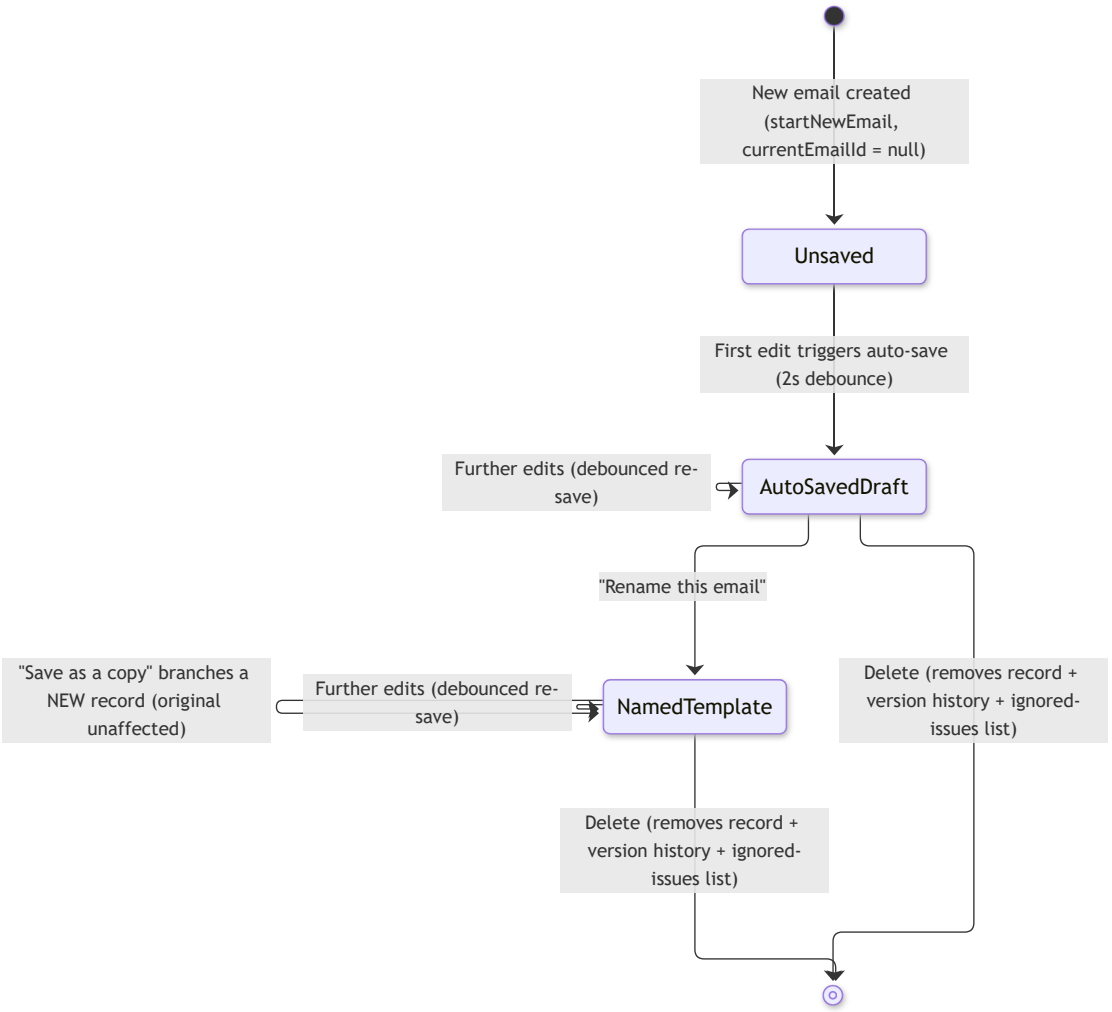
Rule ID	Description	Condition	Action	Affected Modules	Priority	Source
BR-23	Dark-mode token-vs-literal rule	A color's value differs between a light-brand resolve and a dark-brand resolve	It came from a brand token → gets a CSS class; if identical in both passes, it was hand-typed → stays exactly as typed; a manual per-block/row override beats both	Dark mode	High	`dark.ts`
BR-24	Preset re-theming vs. baked-in name	A preset is applied, then the brand is switched later	Colors/fonts (tokens) re-theme; the brand **name**, interpolated into prose once at apply time, does not change	Presets	Medium	`presets.ts`
BR-25	Preset/saved-block id freshness	A preset is applied, or a saved block is inserted	Fresh ids are minted every time (`uid()` / `cloneWithNewIds()`) — the inserted/applied copy can never alias the source	Presets, Saved blocks	High	`presets.ts`, `blocks-library.ts`
BR-26	Slot-derivation render-gating	An AI-fillable "slot" is derived from a block's default data	The slot is exposed only if the same condition that gates the block's actual **rendering** of that field is also true (e.g., hero cards only when `layout` is `cards-2`/`cards-3`)	AI brief generation	Critical	`slots.ts` (documented in-code as a fixed bug — see §28)

Rule ID	Description	Condition	Action	Affected Modules	Priority	Source
BR-27	User-supplied CTA beats AI	`fields.ctaLabel` is non-empty in a brief	The corresponding `cta_label` slot(s) are dropped from the AI schema entirely; the user's own label/URL are written on afterward	AI brief generation	High	`create-from-brief.ts`
BR-28	Brief validation is binary	The AI's JSON response is parsed	The whole response is accepted or the whole thing is rejected with a reason — no field-by-field partial repair	AI brief generation	Critical	`brief-validate.ts`
BR-29	Fallback trigger set is narrow	An AI task call fails	Only `network`/`timeout`/`rate-limit` failures advance to the next configured fallback; any other failure (bad key, no credit, content-policy, bad response) stops immediately	Model routing	Critical	`run-task.ts`
BR-30	Replacement API key must prove itself	A user enters a replacement API key	The new key is only persisted after "Test and save" passes; a failed test leaves the previously working key untouched	AI key management	High	`AiImagePanel.tsx: SettingsView`

14. Workflow States

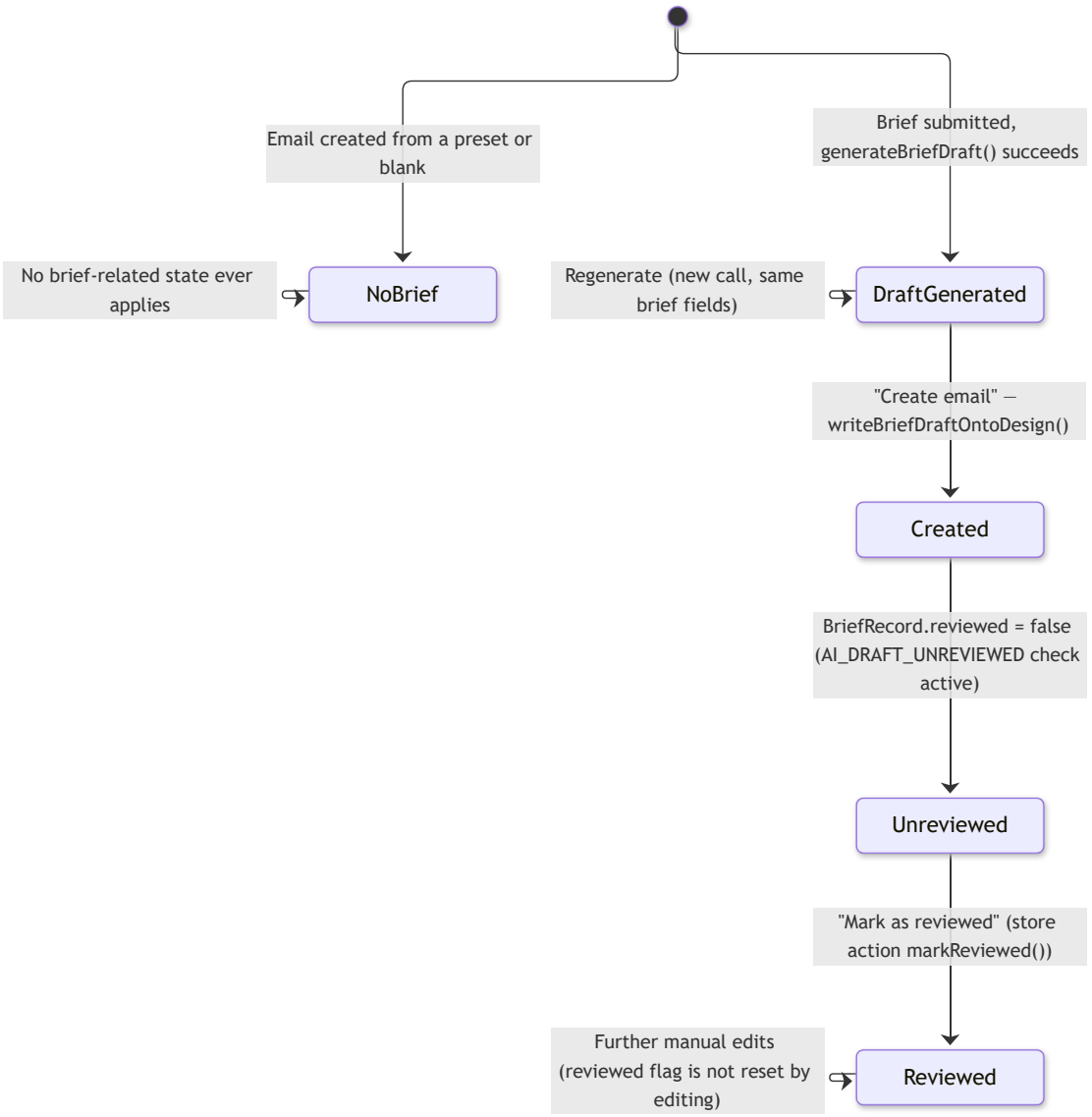
The product has **two independent state machines** worth documenting: the saved-email lifecycle, and the AI-brief review lifecycle. **Not Found in Codebase:** any multi-party approval workflow with states like "Pending Approval"/"Rejected" — there is no such concept anywhere in the product.

14.1 Saved-email lifecycle



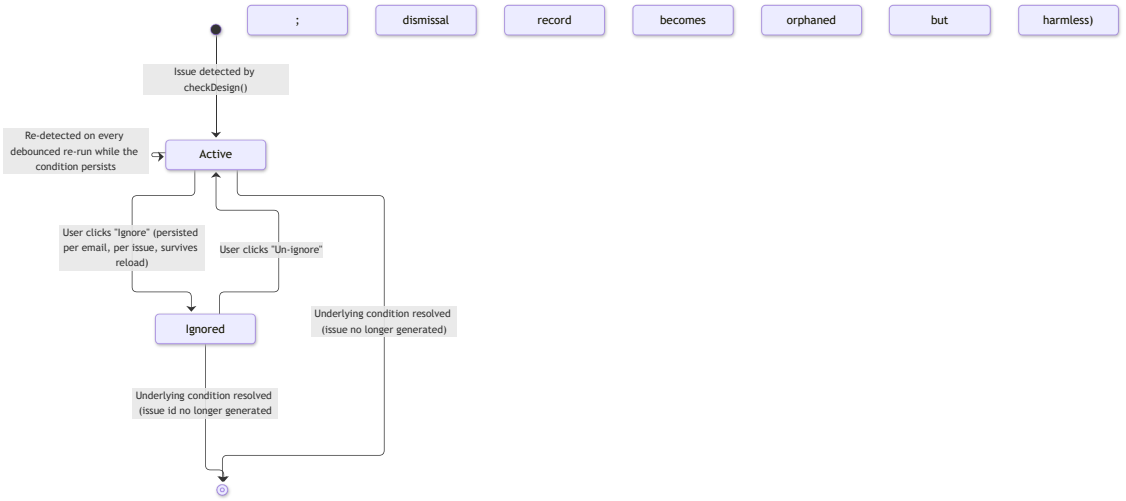
- **Entry conditions:** `Unsaved` on "New"/"Start new email"/"Blank"; `AutoSavedDraft` on the first design mutation after that.
- **Exit conditions:** `AutoSavedDraft` → `NamedTemplate` only via an explicit rename with a non-empty name; any state → deleted via explicit "Delete" + confirm.
- **Allowed actions in any state:** edit, export, preview, save a version snapshot, generate an AI image/brief.
- **Restricted actions:** none are role-restricted (single role); "Rename" is disabled with no record yet ("Make an edit first — this email has no record yet").
- **Transitions are never silent on failure:** a write failure at any transition surfaces as a save-status error ("Not saved: storage full"), never a swallowed exception.

14.2 AI brief draft lifecycle (per email)



- **Entry condition for `Unreviewed`:** exclusively via the "Create email" action on a `BriefReview` screen — there is no path to this state except through a human having seen the review screen first.
- **Exit condition to `Reviewed`:** exclusively a deliberate user click on "Mark as reviewed" (surfaced from the Checks drawer, since `AI_DRAFT_UNREVIEWED` is itself a Checks issue whose "action" *is* that click).
- **Restricted actions:** none — a `Reviewed` email can still be freely edited; there is no "locking" after review.
- **Not Found in Codebase:** any state reversion (e.g., re-flagging as unreviewed after a later edit) — once reviewed, a brief-sourced email stays `Reviewed` regardless of subsequent changes.

14.3 Quality-issue (Checks) per-issue lifecycle



- Issue identity is **stable** (`${code}#${blockId}#${discriminator}`) across unrelated edits, which is what makes "Ignored" meaningful rather than coincidental.

---

15. Data Requirements

Business entities (full field-level dictionary in §16): `Design`, `Row`, `Column`, `Block` (13 types), `Brand`, `SavedTemplate`, `SavedBlock`, `LibraryImage`, `VersionRecord`, `BriefRecord`, plus configuration-only records (`EmailDefaults`, `CurrentUser`/profile, `TaskRoutes`, AI key store, AI text-consent, ignored-issues, merge-tag sample data, session pointer).

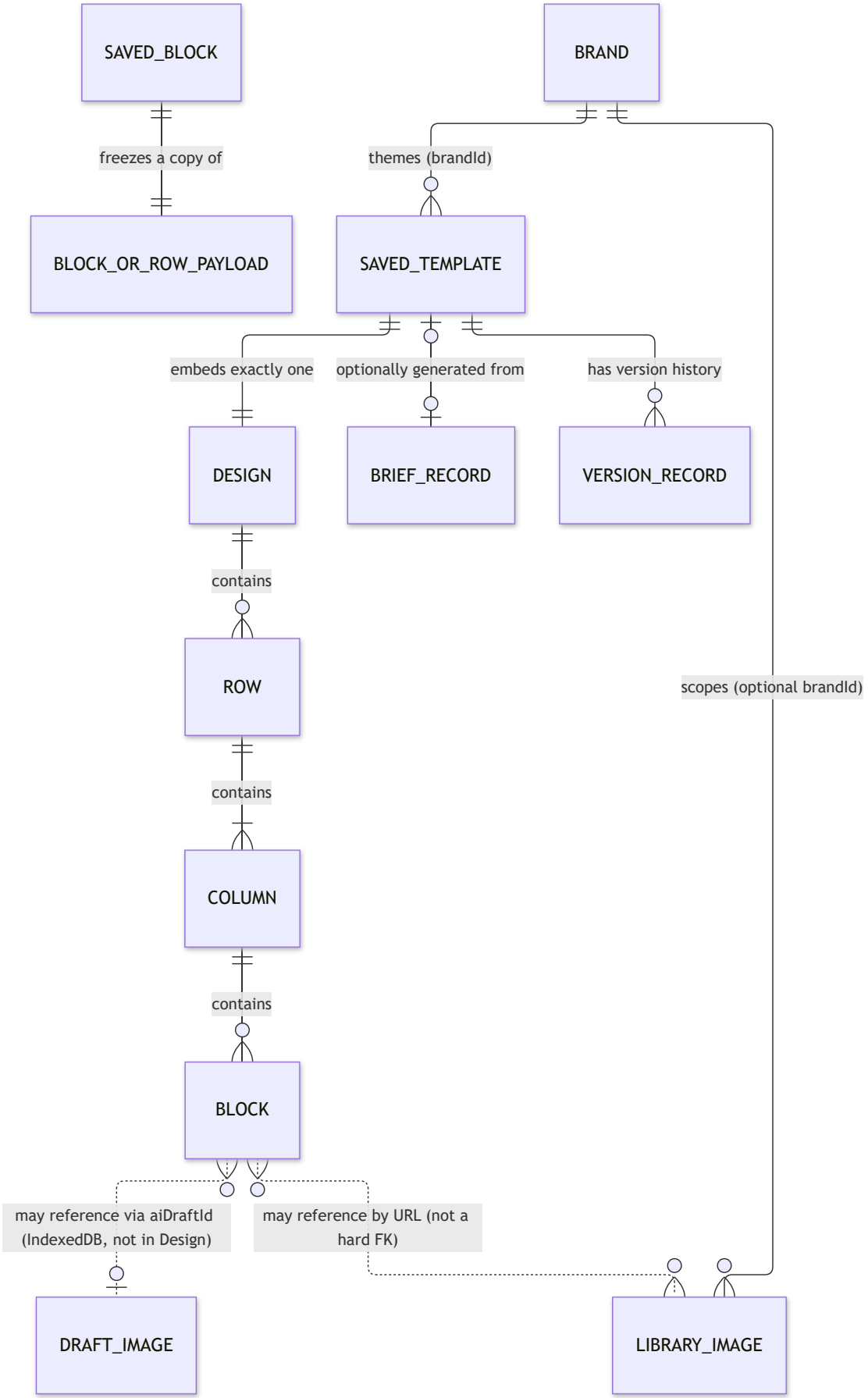
Master data (Inference — relatively slow-changing, team-level): `Brand` kits, the `LibraryImage` library, the `SavedBlock` library, `EmailDefaults`. These are shared across every email in the browser but are **not** shared across browsers/devices except by manual JSON export/import.

Transactional data (Inference — per-document, created/edited routinely): `SavedTemplate` (one per email, including its embedded `Design` and optional `BriefRecord`), `VersionRecord` snapshots, AI-generated image drafts (`DraftImage` in IndexedDB), ignored-issue lists (per email).

Reference data (Fact, fixed in source code, not user-editable): the 8 starter presets (`presets.ts`), the merge-tag ESP-format table (`merge.ts`), the Checks rule catalogue (`checks.ts`), the AI image style presets (`styles.ts`), the social-network enum (11 networks), the font-stack list.

Mandatory vs. optional fields (selected, full list in §17): `Brand.name` mandatory to save; `BriefFields.topic` mandatory to generate; `SaveBlockDialog` name mandatory; every other field across the product defaults to an empty/sane value rather than blocking a save — the product favors "save with defaults" over "block the save."

Relationships:


SAVED_BLOCK
BRAND
freezes a copy of
themes (brandId)
BLOCK_OR_ROW_PAYLOAD
SAVED_TEMPLATE
embeds exactly one
optionally generated from
has version history
DESIGN
BRIEF_RECORD
VERSION_RECORD
contains
ROW
scopes (optional brandId)
contains
COLUMN
contains
BLOCK
may reference via aiDraftId (IndexedDB, not in Design)
may reference by URL (not a hard FK)
DRAFT_IMAGE
LIBRARY_IMAGE

Ownership: every record is implicitly "owned" by whichever browser profile holds it; the `createdBy`/`updatedBy` string fields (Brand, SavedTemplate) are attribution labels only, not access-control ownership (Fact, §5).

Retention: unbounded for templates, brands, saved blocks, and images (**Inference** — no count cap found in code, only reactive quota-pressure pruning of version history); version history explicitly capped at 20 per email; AI model-access-denial flags and AI task "connected this session" status are **not persisted** at all (session-only, cleared on reload); merge-tag sample data and AI text-consent persist indefinitely until "Clear all local data."

Lifecycle: see §14.1 for `SavedTemplate`; `LibraryImage`/`SavedBlock`/`Brand` have no formal lifecycle beyond create → edit → delete (brand: create → edit → archive/restore, a soft-delete, never a hard delete while referenced).

---

16. Business Data Dictionary

16.1 `Design` (the email document)

Field	Meaning	Validation	Notes
`version`	Schema marker	Always `1` today	Single migration seam if a v2 is ever introduced
`brandId`	Which brand themes this design	`string | null`	`null` or a dangling id → falls back to default brand
`body`	Email-wide settings (see 16.2)	—	—
`rows`	Ordered layout	Array, may be empty	—

16.2 `BodySettings`

Field	Meaning	Validation / Default
`title`	The subject line	Checked by `SUBJECT_EMPTY` / `SUBJECT_LONG` (>60 chars)
`preheader`	Inbox preview text	Checked by `PREHEADER_EMPTY` / `PREHEADER_LONG` (>110 chars)
`width`	Email content width (px)	Default 600
`showFooter`	Hide Footer blocks from preview/export	Default `true`
`mergeFormat`	ESP tag spelling at export	`curly` (default) | `mailchimp` | `hubspot` | `custom`
`mergeCustom`	Custom prefix/suffix	Used only when `mergeFormat === 'custom'`
`utm`	UTM tagging settings	`enabled` default `false`
`darkMode`	Whether any dark CSS is emitted at all	Default `true` for new designs; pre-3B designs migrate to `true` as well

16.3 `Row`

Field	Meaning
`layout`	Column ratios, e.g. `[1,2]`
`bgColor`	The **only** background that can span the full window width

Field	Meaning
`contentBgColor`	Background within the content container
`stackOnMobile`	Whether columns stack under the shared mobile media query
`dark`	Optional manual per-row dark override

16.4 `Block` (13 types, discriminated by `type`)

Type	Category	Business-facing purpose
`heading`	Atomic	H1–H6 text
`text`	Atomic	Rich text (TipTap editor)
`image`	Atomic	A single image
`button`	Atomic	A bulletproof CTA button
`divider`	Atomic	A horizontal rule
`menu`	Atomic	A horizontal link list (nav)
`social`	Atomic	Social network icon links
`video`	Atomic	A clickable video thumbnail (email clients cannot play video inline)
`html`	Atomic	Raw HTML escape hatch
`header`	Composite (1B)	Logo + nav, built from Menu
`hero`	Composite (1B)	Eyebrow/heading/subtext/image/CTA, 5 layout variants, built from Button
`footer`	Composite (1B)	Logo/address/legal/unsubscribe/social, built from Social
`articles`	Composite (1B)	Up to 10-item repeater, 3 layout variants

Every block carries `padding` and an optional `dark` override; several carry `noTrack` (per-link UTM opt-out) and/or `aiDraftId` (pointer to a locally-generated image — never image bytes).

16.5 `Brand`

Field	Meaning	Validation
`name` / `slug`	Identity	Name required to save
`logo` / `logo_dark`	Uploaded preview-only logos	≤2MB, downscaled to 400px wide, png/jpeg/svg+xml/webp

Field	Meaning	Validation
`logo_url` / `logo_dark_url`	Hosted logos — the **only** thing exports use	Must be https
`tokens.colors`	9 named roles: primary, secondary, accent, text, muted, link, page_bg, content_bg, border	—
`tokens.fonts`	Heading/body font stacks	From a fixed email-safe list
`tokens.button`	Radius, padding, weight, uppercase	—
`darkTokens`	8-field dark palette, or `null` to derive one	Derivation is key-by-key fallback-safe
`voice`	`description`, `wordsToUse[]`, `wordsToAvoid[]`, or `null`	Consumed only by AI brief generation (6A)
`footer`	Address, legal text, unsubscribe text	—
`social_links`	Repeater of `{network, url}`	`network` from an 11-value enum
`is_active`	Archived (soft-deleted) flag	Archived brands still resolve for old designs
`createdBy` / `updatedBy`	Attribution label (local display name)	Nullable; future-proofing seam, not access control

16.6 `SavedTemplate` (a saved/auto-saved email record)

Field	Meaning
`autoSaved`	`true` until explicitly renamed — groups under "Drafts (auto-saved)"
`brief`	`BriefRecord | null` — set once if this email was generated from a brief
`brandId`	Which brand this email is themed with
`design`	The full `Design` document

16.7 `BriefRecord`

Field	Meaning
`fields`	The exact `BriefFields` submitted
`resolvedPresetId`	The concrete preset used, even if the user chose "Auto" (recorded once, never re-derived)
`reviewed`	Human-review flag (drives `AI_DRAFT_UNREVIEWED`)
`generatedAt`	Timestamp

16.8 `SavedBlock`

Field	Meaning
`kind`	`'block' | 'row'`
`payload`	A frozen JSON snapshot — later canvas edits can never reach it
`category`	Free text; empty groups under "Uncategorised"

16.9 `LibraryImage`

Field	Meaning	Validation
`url`	The image URL	Must be `https://`; `http:` / `data:` / `blob:` /malformed all rejected with a specific reason
`width` / `height`	Natural size	`0` = unmeasured (never guessed)
`brandId`	Optional scope	`null` = shared across every brand
`status`	`'ok' | 'broken' | 'unknown'`	Only set by an actual load probe

16.10 `VersionRecord`

Field	Meaning
`reason`	`'manual' | 'auto' | 'pre-restore' | 'pre-replace'`
`design`	Full snapshot — **never** includes brand definitions

16.11 Owners/usage summary

Entity	Business owner (Inference)	Primary module	Related modules
Design	The editing user	Canvas/store	Render, Export, Checks, Versions
Brand	The editing user (brand owner role, §4)	Brands modal	Presets, Dark mode, AI brief (voice)
SavedTemplate	The editing user	Templates modal/store	Versions, Checks (per-email ignore list)
LibraryImage	The editing user	Image library	Every image-bearing block field, Checks
SavedBlock	The editing user	Saved blocks tab	Canvas (insert)

Entity	Business owner (Inference)	Primary module	Related modules
BriefRecord	The editing user (set once by AI)	Brief generation	Checks (`AI_DRAFT_UNREVIEWED`)

---

17. Validation Rules

Field / Input	Rule	Source
Image/logo URL (library, brand logo_url fields)	Must be `https://`; `http:`, `data:`, `blob:`, empty, or malformed rejected with a specific reason string	`image-storage.ts: validateImageUrl`
Brand logo upload (file)	Type must be png/jpeg/svg+xml/webp; size ≤2MB; bitmaps downscaled to 400px wide; SVG kept verbatim	`image.ts: readLogoFile`
Generic content-image upload	Always rejected — no hosting configured	`image-storage.ts: UploadUnavailableError`
href/src URL scheme (render-time)	Only `https?:`, `mailto:`, `tel:`, `#`, `//`, `{{`, `*|` schemes, or a `` pass; everything else stripped to `''`	`util.ts: safeUrl`
Display name (profile)	Trimmed, capped at 40 chars; empty resets to "Guest"	`user.ts`
Saved-block/row import	Must normalize as a currently-understood `Block` / `Row` shape; unrecognized entries rejected	`blocks-library.ts`
Image-library import	Same pattern via `validateImageUrl`	`image-storage.ts`
Backup file import	Must be a JSON object with all 5 list fields as arrays; after normalization at least one of brands/templates/savedBlocks/images must be non-empty, or the whole import is rejected	`backup.ts: validateBackupShape`
Email size (export)	Soft warn ≥90KB, hard error ≥102KB, measured in UTF-8 bytes of the real rendered HTML	`checks.ts`
Subject/preheader length	Subject >60 chars info; preheader >110 chars info; either empty is warning/error respectively	`checks.ts`
Contrast	WCAG AA — 4.5:1 normal text, 3:1 large text (≥24px, or ≥18.66px bold)	`contrast.ts` / `checks.ts`
Text size / tap target	<12px text → warning; <44px effective button height → warning	`checks.ts`
Link classification	Empty / `#` / relative / `http://` / known placeholder domain, each a distinct rule	`checks.ts: classifyLink`

Field / Input	Rule	Source
Article-list items	Hard cap of 10, enforced at every consumer via `.slice(0, 10)`	multiple files
Brief: topic	Required (non-empty)	`BriefForm.tsx: briefError`
Brief: total length	≤4000 chars (`BRIEF_MAX_CHARS`), summing topic+audience+keyPoints+ctaLabel +sourceText	`brief-types.ts`
Brief: key points	≤8 (`KEY_POINT_MAX`)	`brief-types.ts`
Brief: source text	≤3000 chars (`SOURCE_TEXT_MAX_CHARS`)	`brief-types.ts`
Brief: CTA URL	Must start `https://` if supplied	`BriefForm.tsx`
Brief: AI response — subject options	Non-empty string array, each ≤60 chars after sanitizing	`brief-validate.ts`
Brief: AI response — preheader	≤110 chars	`brief-validate.ts`
Brief: AI response — slots/repeaters	Every declared slot present (string, or null if optional) and within its character budget; every repeater has **exactly** the expected item count	`brief-validate.ts`
Brief: AI response — merge tags	Only `first_name`/`last_name`/`company` allowed through from AI output; any other merge-tag-looking token is stripped	`brief-validate.ts`
AI image: response bytes	PNG/JPEG/WebP identified by magic bytes only (never declared content-type); SVG refused outright; max 10MB	`validate.ts` (lib/ai)
Save Block dialog: name	Required (non-empty, trimmed)	`SaveBlockDialog.tsx`
Brand: name	Required to save	`BrandsModal.tsx`
Data & storage: clear-all	Must type the literal word `DELETE` before the action is enabled	`DataSection.tsx`

18. Notifications

There is **no email/SMS/push notification system** — "Notifications" in this product means **in-app, ephemeral UI feedback** only. **Not Found in Codebase:** any outbound notification channel, scheduling, retry, or escalation mechanism.

Trigger	Recipient	Message (representative)	Channel	Timing	Retry / Escalation
Auto-save completes/fails	The editing user	"Saving…" → "Saved 5 min ago" / "Not saved: storage full"	Top-bar status text	Immediate, ticks over every 30s	None — user must act (e.g., export as JSON, free up space)
Any toast-worthy action (import, rename, delete, insert)	The editing user	e.g. "Imported 3 blocks", "Deleted "Welcome""	Bottom-of-screen toast	~2.6s auto-dismiss	None
Quality issue detected	The editing user	Per-issue message + hint, e.g. contrast ratio + suggested fix color	Top-bar badge + Checks drawer	Debounced 500ms after any change; also on whole-design load	Persists until the underlying condition resolves or the user Ignores it
AI generation failure	The editing user	Classified, redacted error message + diagnostic detail (status/content-type/timing/params, no secrets)	Inline error card in the AI panel / inline banner in the brief form	Immediate on failure	No automatic retry (BR-06); user may Regenerate or rely on a configured fallback
Model access denied	The editing user	"No access to `<model>`" + hint to choose another model or adjust OpenAI project settings	Status chip in Settings → AI	Set on the failing call; cleared on the next success	Persists across reloads only in the sense that re-selecting the same model re-triggers the check from the live record — the flag itself is session-only
Storage nearing capacity	The editing user	"Running low — export emails you no longer need as JSON, then delete them."	Templates modal footer / Data & storage bar	On open of either screen	None automatic — manual cleanup required

Trigger	Recipient	Message (representative)	Channel	Timing	Retry / Escalation
Backup import ready to apply	The editing user	Record-count summary ("Found 2 brand(s), 5 email(s)...")	Inline panel in Data & storage	Immediately after a valid file is chosen	User must explicitly confirm "Review and overwrite"
Destructive action pending	The editing user	Confirmation dialog text (varies by action)	Modal confirm dialog	On the triggering click	User must explicitly Confirm or Cancel

19. Reports

Not Found in Codebase: any report in the traditional BI sense (no saved report definitions, no filters/columns/export-to-CSV reporting feature, no scheduling). The closest analogues, documented for completeness:

"Report" (Inference label)	Purpose	Filters	Columns/Content	Users	Export formats	Scheduling
Storage usage readout	Show how close to the ~5MB local-Storage budget the browser is	None	Bytes used / approximate limit, with a warning past 80%	Editor	None (display only)	None — computed live on open
Checks summary (Export dialog)	One-line severity rollup before export	None	Error/warning/info counts	Editor	None (hands off to the Checks drawer for detail)	Live, on every open
Backup import summary	Preview what a backup file would overwrite before committing	None	Counts of brands/templates/savedBlocks/images	Editor	None	On file selection
Export dialog's UTM line	Reconciliation of link tagging	None	"UTM added to N links, skipped M"	Editor	None	Live, on every open

20. Dashboards

Not Found in Codebase: any dashboard (no widgets, charts, KPIs, or metrics visualization anywhere in `src/`). The top bar's **Checks badge** is the closest functional analogue to a single-metric "status tile":

Element	Widget type (Inference label)	Metric	Role visibility	Refresh	Drill-down
Checks badge	Status chip	Worst outstanding severity (error count, else warning count, else a green check)	Editor (only role)	Debounced 500ms on every design/brand change; also on whole-design load	Click opens the Checks drawer (full issue list)

No other chart, metric, or role-based dashboard view exists.

Part 3 of 4 — Sections 21–30

21. Integrations

Exactly one real, live integration exists: OpenAI, called **directly from the browser** with a user-supplied API key. There is no backend, so there is no server-to-server integration of any kind. (Fact, confirmed by `vite.config.ts` having no `server.proxy`/`server` key at all, and by the AI adapters' own CORS-preflight-driven design.)

Integration	Business purpose	Data exchanged	Trigger	Frequency	Direction	Dependencies	Error handling	Fallback
OpenAI Images API (`POST /v1/images/generations`)	Generate an on-brand image from a text prompt	Sent: prompt (+ style suffix + optional brand color names/hex), size, quality, model, `n:1`, `output_format: 'png'`. Received: base64 PNG/JPEG/WebP bytes. **Never sent:** the app's own API key storage format, logos, footer data, other saved content.	User clicks Send in the AI image panel	On demand, one request per click, manually retried (no auto-retry)	Outbound request / inbound image bytes	A user-held, user-entered API key; a live internet connection	Closed-set error classification (`invalid-key`, `forbidden`, `no-credit`, `rate-limit`, `content-policy`, `network`/`timeout` via gateway statuses, `bad-response`, `cancelled`); all provider-echoed text passed through key-redaction before display	A user-configured fallback model/provider chain (6B), triggered only for `network`/`timeout`/`rate-limit` failures

Integration	Business purpose	Data exchanged	Trigger	Frequency	Direction	Dependencies	Error handling	Fallback
OpenAI Chat Completions API (`POST /v1/chat/completions`, strict JSON-schema mode)	Generate brief-driven email copy (subject/preheader/slot text/image-prompt suggestions)	Sent: brand name + brand voice guidance + the brief's own fields + a "slot guide" (field ids and character limits). **Never sent:** the API key itself, logos, footer/address data, any other saved email content. Received: a strict-schema JSON object.	User clicks "Generate"/"Agree and generate"/"Regenerate"	On demand, one request per click	Outbound request / inbound JSON	A user-held API key; one-time per-provider consent (separate from image-generation consent)	Same closed-set classification as above, plus a model-access-denial signal (`model_not_found`/403-style responses)	Same 6B fallback-chain mechanism
OpenAI `GET /v1/models`	Cheap "does this key work" / "is this model listed" check	Sent: the API key only. Received: a model-id list.	"Test connection" (image tasks); "Add/Replace key" save flow	On demand	Outbound/inbound	A user-held API key	`ok` / `unverified` ("Key OK, model access not verified until first use") / `error`	None needed — this call never bills

Not an integration, but worth stating explicitly: the app's own "Export" is **not an integration with any ESP**. The README states plainly: *"No sending: export the HTML and send it through your ESP (SES, SendGrid, ...)."* There is no SMTP, no ESP API call, no webhook, anywhere in the codebase.

---

22. External Systems

External system	Role	Business dependency	Integrated?
OpenAI	AI text + image generation provider	Core to Features 5A/6A/6B; the product is functionally a design tool without it (every other feature works with no AI configured)	Yes — direct browser-to-API, BYO key (see §21)
ESP (e.g., Amazon SES, SendGrid, Mailchimp, HubSpot)	Where the exported HTML is actually sent from	The entire reason the export/merge-tag/UTM features exist — but the ESP itself is never called by this app	**No** — manual copy/paste or file download is the handoff; this is a deliberate, explicit, current-state limitation, not a bug
Identity Provider / SSO	Would authenticate users	None — no login exists	**No** — Not Found in Codebase
ERP / CRM / HRMS / Payroll	Typical enterprise systems	None referenced anywhere	**No** — Not Found in Codebase
Email/SMS gateway	Would send notifications	None — all "notifications" are in-app UI only (§18)	**No** — Not Found in Codebase
Cloud image hosting / CDN	Would host AI-generated images for real sends	Explicitly **absent by design** — `ImageHosting.configured === false`; this is the reason AI-generated images always export as a placeholder until the user manually hosts and re-pastes a URL	**No**, and the absence is itself a documented, deliberate architectural fact, not an oversight
Payment gateway	Billing	None — the app has no billing of its own; OpenAI bills the user directly for their own API usage, entirely outside this app's visibility	**No**

23. Security Requirements

Authentication: Not Found in Codebase. There is no login, no session, no password, no OAuth, no JWT, no server-side identity of any kind. `src/lib/user.ts`'s own header comment states this is "an internal tool's substitute for an account" — a free-text local display name, never verified, never required, never transmitted anywhere. (Fact, corroborated by an exhaustive grep for `auth|login|password|session` across `src/` turning up only AI-provider-key and in-tab AI-session terminology — see `data-model-and-persistence.md` research sheet.)

Authorization/RBAC: Not Found in Codebase as functioning logic. `canEditBrand(user, brand)` and `getAccessibleBrands(user, brands)` exist as **explicit future-proofing stubs** (1A) that currently return `true`/"every active brand" unconditionally — they are documented in-code as seams for a role system that does not exist today, not as a partially-built feature.

Role management: Not Found in Codebase. Exactly one functional role (§5).

Approval hierarchy: the only approval-like gate is the human "Mark as reviewed" step on an AI-generated brief draft (§13, BR — the closest the product gets to a sign-off), performed by the same single role that generated the draft. **Not Found in Codebase:** any second-person or hierarchical approval.

Sensitive data — the API key is the one genuinely sensitive artifact in this product, and its handling is the most security-engineered part of the codebase:

- Defaults to **in-memory only** (dies when the tab closes); persistence to `localStorage` is strictly opt-in ("Remember on this device"), per provider.
- **Deliberately excluded from the zustand store** specifically so no future feature that serializes app state (export, backup, saved blocks) can accidentally leak it — enforced by dedicated tests sweeping every such surface (design, brands, saved blocks, image library, issues, rendered export, every localStorage entry, `document.body.textContent`).
- **Deliberately excluded from `backup.ts`'s export**, enforced structurally (that file never imports the key-store module at all) rather than by a runtime filter.
- Displayed only in masked form (`maskKey()`); never rendered into an input the app itself controls once saved.
- Provider error text is passed through `redactKeys()` before ever reaching the UI — strips the exact key, any `sk-`/`rk-`/`org-`/`proj-`-prefixed token, and any `Bearer <token>` pattern, because "a provider is free to echo the request, including the key, back at us."
- **Stated plainly to the user, unencrypted:** the "Remember on this device" checkbox's own copy reads *"Stored unencrypted in this browser, for this provider only."* — **Fact**, and a real, disclosed risk rather than a hidden one.
- No outbound header beyond `Authorization: Bearer <key>` and `Content-Type: application/json` is ever sent — confirmed deliberate, because the live CORS preflight only allows exactly those two headers; sending an org/telemetry header would fail the request outright.

Audit requirements / compliance: Not Found in Codebase. No audit log, no compliance framework reference (GDPR/SOC2/HIPAA etc.), no data-residency statement. See §24 for the (much smaller) "audit trail" that does exist.

Password policy: Not Found in Codebase (no passwords exist).

Session policy: the only "session" concept is the in-memory lifetime of an AI provider API key and its "Connected" status, both of which end when the tab/browser closes. **Not Found in Codebase:** any app-level session timeout/expiry.

Access restrictions: none beyond "only someone with physical/browser access to this machine's profile can see or edit anything," since all data is local and unauthenticated.

Prompt-injection defense (AI brief generation, 6A) — a genuine, two-layered security control worth calling out as such:

1. The brief and any pasted source text are wrapped in explicit `===== BEGIN/END (untrusted data) =====` delimiters with a system-prompt instruction to treat their contents as data, never instructions, even if they contain apparent commands.
2. A mechanical backstop: the model's response is constrained to a strict JSON Schema (`additionalProperties: false`, every field required-or-nullable) — injected text cannot smuggle extra fields into the response regardless of what it asks for.

Response-content validation (both AI pipelines) is itself a security boundary: AI image bytes are sniffed by magic number, never trusted declared content-type, and SVG is refused outright specifically because "it is a document that can carry script, not an image." AI text output passes through `brief-validate.ts`'s sanitizer (HTML tags stripped, markdown stripped, bare URLs stripped) before ever being written into a design that will later be rendered as HTML.

---

24. Audit Trail

What is tracked (Fact):

Tracked fact	Where	Granularity
`createdAt` / `updatedAt` timestamps	`Brand`, `SavedTemplate`	Per record
`createdBy` / `updatedBy` display-name label	`Brand`, `SavedTemplate`	Per record, stamped from the local Profile name at the moment of the write
Full design-state history	`VersionRecord` (version history)	Up to 20 snapshots per email, with a reason (`manual` / `auto` / `pre-restore` / `pre-replace`) and an optional free-text note
Undo/redo history	In-memory `past` / `future` stacks	Up to 100 steps, cleared on reload (not persisted)
Which AI brief a given email came from, and whether it has been reviewed	`BriefRecord` on the `SavedTemplate`	Per email

What is NOT tracked (Not Found in Codebase): a changelog of *what specifically changed* between versions (version history stores full snapshots, not diffs); who approved what (no second-person approval exists); any log of AI prompts/responses actually sent (the brief's own fields are stored, but the raw request/response payload to OpenAI is not persisted anywhere); any login/access log (no login exists); any export/send log (the app never sends anything itself).

Compliance requirements: Not Found in Codebase. No regulatory framework is referenced.

25. Non-Functional Requirements

Category	Finding	Source
Performance	Checks re-run debounced at 500ms; auto-save debounced at 2s; AI calls self-throttled with a 2-second minimum gap between calls for the same task and a per-task one-in-flight guard; a 90-second client-side timeout on AI image requests. **Not Found in Codebase:** any load-testing artifact, performance budget beyond the ~90/102KB email-size checks (which are an *output quality* rule, not an app-performance one).	`checks.ts`, `output.ts`, `provider.ts`
Availability	Entirely client-side — availability equals "the user's own browser works and `api.openai.com` is reachable when AI features are used." No server uptime to manage. **Inference.**	Architecture
Scalability	Single-browser, single-user by design; there is no concept of "more users" to scale to. Internally, the one acknowledged scale concern is storage: `localStorage` is a fixed ~5MB budget per origin, with version-history pruning as the only automatic relief valve — templates/brands/images/saved blocks are explicitly **unbounded in count** with no automated pruning (**Inference**, confirmed by the absence of any cap in the quota-recovery code path for those categories).	`storage.ts`
Security	See §23 in full.	—
Reliability	Every storage write failure is surfaced to the user, never silent (`writeSafe()`'s `'ok'/'pruned'/'full'` contract, consistently checked at every call site). AI calls never silently retry (to avoid double-billing). A malformed individual record on import/load is dropped rather than failing the whole load.	`storage.ts`, `design-ops.ts`, `backup.ts`

Category	Finding	Source
Maintainability	`tsconfig.json` enables `strict`, `noUnusedLocals`, `noUnusedParameters`, `noFallthroughCasesInSwitch` — directly relevant to `render.ts`'s large block-type switch statement. 39 test files cover rendering, business rules, AI pipelines, and UI integration; `tsc --noEmit` is the only static-analysis gate (**no dedicated lint script exists in the repo**, per the 6B amendment's own closing note).	`tsconfig.json`, `src/test/*`, `docs/PROGRESS.md`
Extensibility	Deliberately designed seams: the AI task registry ("adding a task is one new id and one new `AI_TASKS` entry"); the slot-derivation pattern (a 9th preset is supported automatically); the brand permission functions (`canEditBrand` / `getAccessibleBrands`) as a pre-built seam for a future role system.	`tasks.ts`, `slots.ts`, `brand.ts`
Accessibility	WCAG 2.1 AA contrast checking (text/button/link, light and dark) is a first-class, tested quality rule; `alt` text is enforced as required in the editor for image-bearing composite blocks; tap-target (44px) and minimum text-size (12px) checks exist. **Not Found in Codebase:** any accessibility statement/audit for the *builder UI itself* (keyboard-nav, screen-reader support of the editor) — the accessibility engineering found is entirely about the **output email's** accessibility, not the app's own.	`contrast.ts`, `checks.ts`
Localization	The AI brief generator supports 3 output languages (English, Tamil, Hindi) for generated **content**. **Not Found in Codebase:** any localization of the **application UI itself** (no i18n framework, all UI strings are hardcoded English JSX).	`brief-types.ts`

Category	Finding	Source
Usability	Debounced auto-save with a persistent, honest status line; non-blocking quality checks with "Go to block" jump-to; a single two-level Settings hub; keyboard shortcuts (Ctrl+Z/Shift+Z/Y, Ctrl+D, Delete, Escape) documented in-app under About.	`TopBar.tsx`, `AboutSection.tsx`
Backup	A one-click full-data export (brands, templates, saved blocks, images, versions) as a single JSON file.	`backup.ts`
Recovery	Version history (20/email) + undo/redo (100 steps, in-memory) + auto-save + a pre-restore/pre-replace snapshot rule that makes every restore itself reversible.	`versions.ts`, `store.ts`
Monitoring	**Not Found in Codebase.** No telemetry, error reporting, or usage analytics of any kind exists in `src/`.	—

26. Assumptions

#	Assumption	Status
1	The product is intended for a single internal team (named "Vee MARCOM") rather than a multi-tenant external audience	Fact (1A quote) + Inference (extrapolated scope)
2	A team sharing brand kits/templates across people does so by physically sharing a browser profile or by manual JSON export/import, not by any live sync	Fact (README "Known limitations")
3	The organization has its own OpenAI account/billing and accepts that costs are incurred directly, outside this app's visibility	Inference (architecture + explicit no-cost-estimator finding)
4	Sending the finished email through an ESP is a separate, already-existing process the team owns	Fact (README: "No sending")
5	Users are trusted individually — there is no need to prevent one browser's user from seeing/editing any other data in that same browser	Fact (no RBAC exists)
6	The quality-check thresholds (90/102KB, WCAG AA, 12px/44px) reflect real inbox-client behavior (Gmail clipping, accessibility standards) rather than arbitrary numbers	Inference, consistent with how each is described in-code (e.g., "modeled on Gmail's clipping behavior")
7	A future multi-user/role system is plausible enough to be worth leaving seams for (`createdBy`/`updatedBy`, `canEditBrand`) even though none exists today	Fact (explicit in-code "future-proofing" comments, 1A)

27. Constraints

Constraint	Type	Detail
No backend	Technical	Every feature must work from a static SPA; AI calls must be browser-callable directly (verified via CORS preflight before 5A was even built)
~5MB `localStorage` budget	Technical	Shared across every persisted category except AI image drafts (IndexedDB) and AI keys (also localStorage, but excluded from the quota-recovery conversation in practice since keys are tiny)
Email-client HTML constraints	Technical/business	No flexbox, no grid, no CSS `position`, table-based layout, inline styles, Outlook conditional markup required — because the output must render in real inboxes, not just browsers
Email clients strip `data:` URIs	Technical	Drives the entire "logo preview vs. hosted URL" split (1A) and the entire "no file upload" policy (2C-1)
No image hosting exists	Business/operational	AI-generated images can never ship as real bytes in an export — only as a placeholder — until the team stands up or adopts hosting themselves
BYO API key, no backend proxy	Business	The app can never meter, cap, or centrally bill AI usage; each individual user bears their own OpenAI cost directly
CORS header allow-list is exactly `authorization,content-type`	Technical	No organization/telemetry header can ever be sent to OpenAI from this app, by the provider's own enforcement
OpenAI model lifecycle (models deprecate/rename)	Operational	Already observed twice in the project's own history — `gpt-image-1` stopped existing; `gpt-4.1-mini` momentarily considered stale — model lists must be kept current by whoever maintains the code
No dedicated lint tooling	Technical/process	`tsc --noEmit` is the only static-analysis gate (explicitly noted as a fact, not treated as a problem, in the project's own history)

28. Risks

Risk	Category	Description	Mitigation present today
Total data loss on a lost/wiped/corrupted browser profile	Data	Everything lives in one browser's local storage with no automatic cloud backup	Manual "Export all data" backup exists, but is **not automatic or scheduled** — a user who never exports loses everything if the browser profile is lost
`localStorage` quota exhaustion	Data/Operational	A heavy user (many large templates/images) can hit the ~5MB ceiling	Automatic version-history pruning is the only automated relief; templates/brands/images/saved blocks have no automated pruning — a user must manually delete things once genuinely full
API key exposure	Security	The key is BYO, unencrypted if remembered, and lives in a browser that could be compromised by other means (malware, shared machine)	In-memory default, explicit "unencrypted" disclosure, redaction in error paths, exclusion from every serialization surface — residual risk is accepted and disclosed, not eliminated
Unreviewed AI content shipping in a real send	Process	`AI_DRAFT_UNREVIEWED` is a **warning, not a block** — a user can export and send an AI-generated draft without ever clicking "Mark as reviewed"	The warning is visible in both the badge and the Export dialog's summary line, but nothing technically prevents ignoring it (consistent with the product-wide "never block" philosophy, §13 BR-05)
AI hallucination / factual error in brief-generated copy	Process	The system prompt forbids inventing facts and requires a bracket placeholder instead, but nothing guarantees the model complies	`BRIEF_PLACEHOLDER_LEFT` surfaces leftover placeholders; human review is strongly encouraged via the unreviewed-warning, but is not enforced
Sending a preset with placeholder art/copy by mistake	Process	Every preset ships `picsum.photos` images and fictional copy on purpose	`IMAGE_PLACEHOLDER` / `LINK_PLACEHOLDER_DOMAIN` checks exist precisely to surface this, but export is never blocked

Risk	Category	Description	Mitigation present today
Model deprecation breaking AI features silently	Integration	OpenAI can rename/retire a model id (already happened once in this project's history)	Hardcoded model lists require a code update when this happens; "Other model id" free-text and the model-access-denial chip provide a partial, reactive safety net, not prevention
Cost overrun from AI usage	Operational	No usage cap, budget, or cost estimator exists anywhere in the app	None — explicitly absent (§3)
Email rendering differences across clients the engine cannot fully model	Output quality	Contrast over a background image is unmeasurable; Outlook-on-Windows ignores certain background techniques; Gmail's own apps ignore all dark-mode CSS	The engine is honest about its limits (`CONTRAST_UNKNOWN`, documented Outlook/Gmail caveats in the Preview UI) rather than guessing
Single point of failure: one browser, one person	Business continuity	If the one person with the browser profile is unavailable, the team's brand kits/templates are inaccessible without a prior backup export	None beyond the manual backup feature

29. Gap Analysis

Gap	Category	Detail
No image hosting	Missing requirement (acknowledged)	AI-generated images can never ship for real without a manual download-and-rehost step; the README's own "Ideas for next versions" lists this
No sending integration	Missing requirement (acknowledged)	The product stops at HTML/plain-text export; no ESP is ever called
No multi-user collaboration or sync	Missing requirement (acknowledged)	Everything is per-browser; the seams (`createdBy` / `canEditBrand`) exist but nothing is built on them
No cost/usage controls for AI	Missing requirement	No cap, estimate, or visibility into OpenAI spend from within the app
No Checks re-run on image-library re-probe	Partially implemented	Editing a brand re-runs Checks automatically; re-probing or adding a library image does not — the next unrelated design edit picks it up (explicitly documented as a known gap in Feature 3A)
No CTA-field targeting in brief generation	Partially implemented / accepted limitation	A user-supplied CTA label/URL applies to **every** `cta_label`-kind slot a preset exposes, not to one specifically chosen button (6A, explicitly documented trade-off)
No cross-panel "use this AI image prompt" handoff	Partially implemented	The brief review screen only offers "Copy" for a suggested image prompt — no one-click "open in AI image panel" (6A, explicitly deferred as non-blocking)
Task-settings staleness across open panels	Technical limitation affecting UX	Changing a task's model in Settings while the AI image panel is simultaneously open does not live-update that panel's visible model name until an unrelated re-render — deliberately not fixed with a shared subscription, as "disproportionate to one rare case" (6B, explicitly documented)
No automated pruning for templates/brands/images/saved blocks	Technical limitation	Only version history is auto-pruned under quota pressure; everything else requires manual deletion once storage is genuinely full

Gap	Category	Detail
No accessibility audit of the builder UI itself	Missing requirement	All accessibility engineering targets the **output email**, not the editor application
No localization of the application UI	Missing requirement	Only AI-generated **content** supports multiple languages; the UI itself is English-only, hardcoded
No automated/CI pipeline evidence	Not Found in Codebase	No CI config file was found in the directories read for this document; `npm test`/`npm run build` exist as scripts but whether they run automatically on push is **Not Found in Codebase** within this review's scope

---

30. Future Enhancements

Sourced primarily from the README's own **"Ideas for next versions"** section, supplemented with Inference drawn from the gaps above.

Enhancement	Priority (Inference)	Business value
Image hosting (so AI-generated and uploaded images can ship as real bytes, not placeholders)	**High**	Removes the single biggest "doesn't actually work yet" gap in the AI-image feature; directly unblocks real sends using AI-generated art
Supabase persistence (auth + a `templates` table with the design JSON in a `jsonb` column) — named explicitly in the README	**High**	Would be the foundation for real multi-user collaboration, cross-device sync, and finally giving the existing `createdBy` / `canEditBrand` seams something real to do
Test-email sending	**Medium**	Closes the loop between "export" and "verify it actually renders correctly" without leaving the tool
Merge-tag picker with groups and display conditions	**Medium**	Improves personalization sophistication for larger contact-field sets
Countdown-timer block	**Low**	A common promotional-email pattern not currently supported
Template-gallery thumbnails	**Low**	Visual browsing improvement for the Templates modal
AI "generate section" (targeted regeneration using the design JSON schema, rather than whole-email brief generation)	**Medium**	Lets a user ask AI to redo just one section of an otherwise-finished email, instead of regenerating or hand-editing
A second real AI provider (the current registry has one unfilled placeholder slot)	**Medium**	Makes the already-built fallback-chain feature (6B) actually useful across providers, not just across models within one provider
Usage/cost visibility for AI calls	**Medium**	Directly addresses the "no cost control" risk in §28
Automated pruning or clearer warnings for templates/brands/images approaching quota	**Low–Medium**	Reduces the risk of an unrecoverable "storage full" moment for a heavy user

Part 4 of 4 — Sections 31–39

31. Acceptance Criteria

Given/When/Then criteria below are **Inference**, phrased from the behavior and tests documented in `docs/PROGRESS.md` and confirmed in source — no formal acceptance-criteria document exists in the repository.

31.1 Brand kit (1A)

```
Given an email using brand.* color tokens
When the active brand is switched
Then every token-sourced color/font re-themes instantly
 And every manually-typed hex override on that email is unchanged
 And design.brandId is the only field that changed

Given a brand has no logo_url set
When the email is exported
Then the export contains no data: URI for that logo
 And the Checks/image-audit warns that the logo is not https, if a preview upload exists but
no hosted URL does
```

31.2 Composite blocks (1B)

```
Given a Footer block exists and Email settings → Show footer is off
When the email is previewed or exported
Then the Footer block produces no HTML in the output
 And the canvas still shows it, dimmed, with a "Hidden on send" badge

Given a Hero block's layout is changed from "centered" to "cards-3"
When the change is applied
Then exactly 3 cards are present, padded with defaults or trimmed from the prior count
```

31.3 Presets (1C)

```
Given the user opens "New email" and picks the "Newsletter" preset
When the preset is applied
Then a fresh Design is created with new ids throughout
 And editing the new email never mutates the shared preset definition
 And the Checks drawer immediately shows IMAGE_PLACEHOLDER and LINK_PLACEHOLDER_DOMAIN
warnings
```

31.4 Merge tags / UTM / plain text (2A)

```
Given UTM tagging is enabled with utm_medium="email"
When the email is exported
Then every qualifying link gains UTM parameters
 And the unsubscribe link is never tagged but IS counted in the "skipped" total
 And a link with its own "Don't add tracking" opt-out is never tagged

Given Email settings → Merge tag format is "Mailchimp"
When the email is exported
Then {{first_name}} becomes *|FNAME|* in the output
 And the stored design content still reads {{first_name}} unchanged
```

31.5 Saved blocks / auto-save / version history (2B)

```
Given a new, never-before-saved email is being edited for the first time
When 2 seconds pass after the first edit
Then a new SavedTemplate record is created, flagged autoSaved: true

Given a version is restored from History
When the restore completes
Then the design present immediately before the restore has itself been snapshotted
 And the user can immediately restore forward again to undo the restore
```

31.6 Image library (2C-1)

```
Given a user attempts "Upload from device" on an Image block
When the file dialog resolves
Then the action is refused with the message
  "Upload needs image hosting. Paste a public https image URL instead."
 And no data: URI is written into the block

Given an image URL has never been checked this session
When the Checks drawer is viewed
Then that image is never reported as IMAGE_BROKEN
 And only becomes reportable after "Check images load" is pressed and the probe fails
```

31.7 Checks engine (3A)

```
Given an email has 1 error and 2 warnings outstanding
When the user opens the Export dialog
Then Copy and Download remain enabled
 And the dialog shows "1 error, 2 warnings" with an "Open checks →" link

Given a Checks issue has been Ignored for a specific email
When the browser is reloaded and that same email is reopened
Then the issue remains ignored (hidden from the active list, visible under "Ignored (n)")
```

31.8 Dark mode (3B)

```
Given Email settings → Dark mode support is off
When the email is exported
Then the output is byte-identical to the same email exported before dark mode existed
 (no meta tags, no dm- class names, no extra <style> block)

Given a brand has no dark palette configured
When dark mode is on and the email is previewed in Dark
Then a derived palette is used (near-black page/panel, link lightened to clear 4.5:1, etc.)
 And no field renders as an unreadable or invisible color
```

31.9 AI image generation (5A)

```
Given no API key is configured
When the user opens the AI image panel
Then it opens directly into the Settings view, not the chat composer

Given a generation request fails with a 503 status
When the error is classified
Then the user sees a warning that the image "may well have been made and billed"
 And no automatic retry is attempted
```

31.10 AI brief generation (6A)

```
Given a brief with topic="Diwali sale" and no CTA label supplied
When the draft is generated
Then every cta_label-kind slot in the schema is AI-fillable
Given the same brief WITH a CTA label supplied
Then those same slots are removed from the AI schema entirely
 And the user's own label/URL are written onto the design directly

Given the AI response contains a slot value exceeding its character budget
When brief-validate.ts checks it
Then the entire response is rejected (not just that one field)
 And "Regenerate" is offered, with nothing written to any design

Given an email was created from a brief
When it is opened
Then the Checks drawer shows AI_DRAFT_UNREVIEWED
 until "Mark as reviewed" is clicked
```

31.11 Per-task AI model routing (6B)

```
Given a task's current model has no configured key
When the user's prior generation on that task fails with a timeout
Then the next fallback entry with a configured, connected key is tried automatically
Given the same failure is instead "invalid key" or "content policy"
Then no fallback is tried — the failure is surfaced immediately

Given a per-generation Quality selector is set to "Draft"
When the task's configured default quality is "High"
Then the generation uses "Draft" — the explicit per-call choice always wins
```

31.12 Settings hub / data governance (7A)

```
Given the "Clear all local data" danger-zone action
When the confirmation text field does not contain exactly "DELETE"
Then the action button remains disabled

Given a backup file is selected for import
When it is shape-validated
Then a summary of record counts is shown
 and nothing is overwritten until the user explicitly confirms
```

---

32. User Stories

All stories have a single actor — the Editor role (§5) — since no other role exists. Each includes acceptance criteria referencing §31 where applicable.

1. **As an** editor, **I want** to switch which brand an email uses **so that** I can re-theme a campaign for a different sub-brand without rebuilding it. *(AC: §31.1)*
2. **As an** editor, **I want** to start from a ready-made preset **so that** I don't begin every email from a blank canvas. *(AC: §31.3)*
3. **As an** editor, **I want** the app to warn me about placeholder art and copy left in a preset **so that** I never accidentally send unfinished content. *(AC: §31.3, §31.7)*
4. **As an** editor, **I want** my work auto-saved as I go **so that** closing the tab by accident never loses anything. *(AC: §31.5)*
5. **As an** editor, **I want** a version history I can preview and restore **so that** I can recover from a bad edit without fear of losing my current state. *(AC: §31.5)*
6. **As an** editor, **I want** to save a block or row I built **so that** I can reuse it (e.g., a CTA band) in future emails without rebuilding it. *(AC: implicit in 2B feature description, §9.5)*
7. **As an** editor, **I want** a centralized, vetted image library **so that** I stop hunting for the same hero image URL across emails. *(AC: §31.6)*
8. **As an** editor, **I want** to be warned — but never blocked — about size, broken links, poor contrast, and missing alt text **so that** I can fix real problems before a send without being stopped from exporting a work-in-progress. *(AC: §31.7)*
9. **As an** editor, **I want** my export to look correct in dark-mode-aware inboxes **so that** I don't have to maintain two separate versions of the same email. *(AC: §31.8)*
10. **As an** editor, **I want** to generate a hero or article image from a text prompt **so that** I don't need a separate design tool for on-brand art. *(AC: §31.9)*
11. **As an** editor, **I want** a clear warning when a generated image isn't actually hosted anywhere **so that** I never accidentally send an email with a broken image. *(AC: §31.9, §9.9)*
12. **As an** editor, **I want** to turn a short content brief into a full draft email **so that** I don't have to write the first draft from scratch. *(AC: §31.10)*
13. **As an** editor, **I want** to review and approve an AI-generated draft before it's final **so that** I catch factual errors or missing details before sending. *(AC: §31.10)*
14. **As an** editor, **I want** to choose a different AI model for text vs. image generation, with a fallback if one fails **so that** I'm not stuck when a provider has an outage or doesn't support a model I want. *(AC: §31.11)*
15. **As an** editor, **I want** one place to manage my profile, AI keys, brand defaults, and data **so that** I'm not hunting across the app for settings. *(AC: §9.12)*
16. **As an** editor, **I want** to back up and restore all my local data **so that** I can move to a new browser/machine without losing my brands, templates, and saved blocks. *(AC: §31.12)*
17. **As an** editor, **I want** a hard-to-trigger-by-accident "clear everything" action **so that** I can start fresh deliberately, without a stray click wiping my work. *(AC: §31.12)*
18. **As an** editor, **I want** my AI API key to never leave this browser except to the AI provider itself **so that** I'm not exposing a billable credential through an export or backup file. *(Implicit AC: §23, §13 BR-08)*

33. Business Glossary

Term	Meaning
Design	The in-memory/persisted document representing one email: body settings + rows + columns + blocks
Brand kit / Brand	A named set of colors, fonts, logos, footer content, social links, and optional "voice" guidance used to theme emails
Resolve at the boundary	The architectural rule that color/font tokens (`"brand.primary"`) are converted to literal values exactly once, immediately before rendering/exporting, never inside the renderer itself
Block	One piece of content in the email — 9 atomic types (heading, text, image, button, divider, menu, social, video, html) and 4 composite types (header, hero, footer, articles)
Composite block	A block type (Header/Hero/Footer/Article list) built internally by reusing the real atomic renderers rather than duplicating HTML logic
Preset	One of 8 fixed starter structures offered in the "New email" dialog, brand-neutral and placeholder-filled on purpose
Checks	The non-blocking email-quality engine; "an issue" (or "Issue") is one finding with a severity, message, hint, and stable id
Severity (error / warning / info)	Checks' three-tier classification; none of the three ever disables Copy/Download/Export
Ignore (a check)	A per-email, per-issue dismissal that persists and survives reload until explicitly un-ignored
Draft (AI image)	A locally generated image's bytes, held in this browser's IndexedDB; the design only ever stores its id, never the bytes
AI draft unreviewed	The specific Checks warning that appears on any email created from a brief until "Mark as reviewed" is clicked
Slot	One AI-fillable text field, derived automatically by walking a preset's real block tree and classifying by block type (the "slot-derivation pattern," 6A)
Task (AI)	One of the product's two AI capabilities — `text.brief` or `image.generate` — each independently routed to a provider/model with its own fallback chain (6B)

Term	Meaning
Fallback chain	An ordered, user-configured list of alternate provider/model choices tried only on a network/timeout/rate-limit failure, never on a bad key or content-policy refusal
Merge tag	A `{{tag}}`-syntax personalization placeholder, stored canonically but reformatted per ESP (Mailchimp/HubSpot/custom) only at export
UTM tagging	Appending campaign-tracking query parameters to outbound links at export time only
Auto-saved draft (in the Templates modal)	A `SavedTemplate` record not yet given an explicit name by the user — distinct from a "named template"
Version / **Version history**	A full snapshot of a design taken manually, automatically (≤1/5min), or immediately before a restore/replace
Saved block	A frozen, reusable copy of a block or row, kept in a searchable per-browser library
Image library	The per-browser catalogue of vetted, name-and-alt-tagged, optionally brand-scoped image URLs
Placeholder image/link	Stock art (`picsum.photos` etc.) or a fake domain (`example.com` etc.) deliberately shipped in every preset, specifically to trip the `IMAGE_PLACEHOLDER`/`LINK_PLACEHOLDER_DOMAIN` checks until replaced
Dark-mode additive guarantee	With dark mode off, export output is byte-identical to a build of the app that never had dark mode at all
Profile (display name)	A local, unauthenticated, free-text label used only for `createdBy`/`updatedBy` attribution — not a login identity
Settings hub	The single two-level configuration surface (7A) covering Profile, AI, Brands, Email defaults, Checks, Data & storage, About
Backup	A one-file JSON export of brands, templates, saved blocks, images, and versions — deliberately excluding AI keys and drafts
Clear all local data	The typed-"DELETE"-gated action that wipes every category above, including AI keys and drafts
BYOK (bring your own key)	The architecture where the user supplies their own OpenAI API key, billed directly to them, with no backend proxy

34. Frequently Asked Questions

How does "approval" work for AI-generated content? There is no multi-person approval. The only gate is a single Checks warning, `AI_DRAFT_UNREVIEWED`, cleared by clicking "Mark as reviewed" — which the same person who generated the draft can click immediately, with or without actually changing anything. It is a reminder, not a control.

Who can edit a record (brand, template, saved block)? Whoever has the browser open. There is no login and no per-record permission check; `canEditBrand()` always returns `true` today.

What happens after an AI generation is "rejected" by validation? Nothing is written anywhere. The user sees the rejection reason and a "Regenerate" button. The brief fields the user typed are untouched, so regenerating doesn't require retyping anything.

How are notifications sent? There is no external notification channel. "Notifications" are in-app only: toasts (auto-dismiss ~2.6s), the top-bar save-status text, and the Checks badge/drawer.

Can two people work on the same email together? Not through this product. Each browser's local storage is independent; the only way to move work between people/machines is manual JSON export/import (whole designs, brand kits, saved blocks, images) or the full Data & storage backup/restore.

What happens if my browser's storage fills up? Writes never fail silently. The app first tries to free space automatically by pruning the oldest version-history snapshots (10 at a time, across every email); if that's not enough, it reports "storage full" so the user can manually export/delete something. Brands, templates, images, and saved blocks are never auto-deleted.

Does the app ever see or store my OpenAI API key insecurely? The key defaults to memory-only (gone when the tab closes). If you opt into "Remember on this device," it's written to `localStorage` — the UI explicitly tells you this is unencrypted. The key is deliberately kept out of the app's main state store and out of the backup export, both enforced structurally and verified by dedicated tests, so no export/backup/saved-block action can leak it.

Will my export look right in dark mode? If "Dark mode support" is on for that email (default: on for new emails), the export carries additive dark-mode CSS that most major clients respect — except Gmail's own apps, which always apply their own automatic color inversion regardless of anything this app does; the in-app Preview explicitly says so.

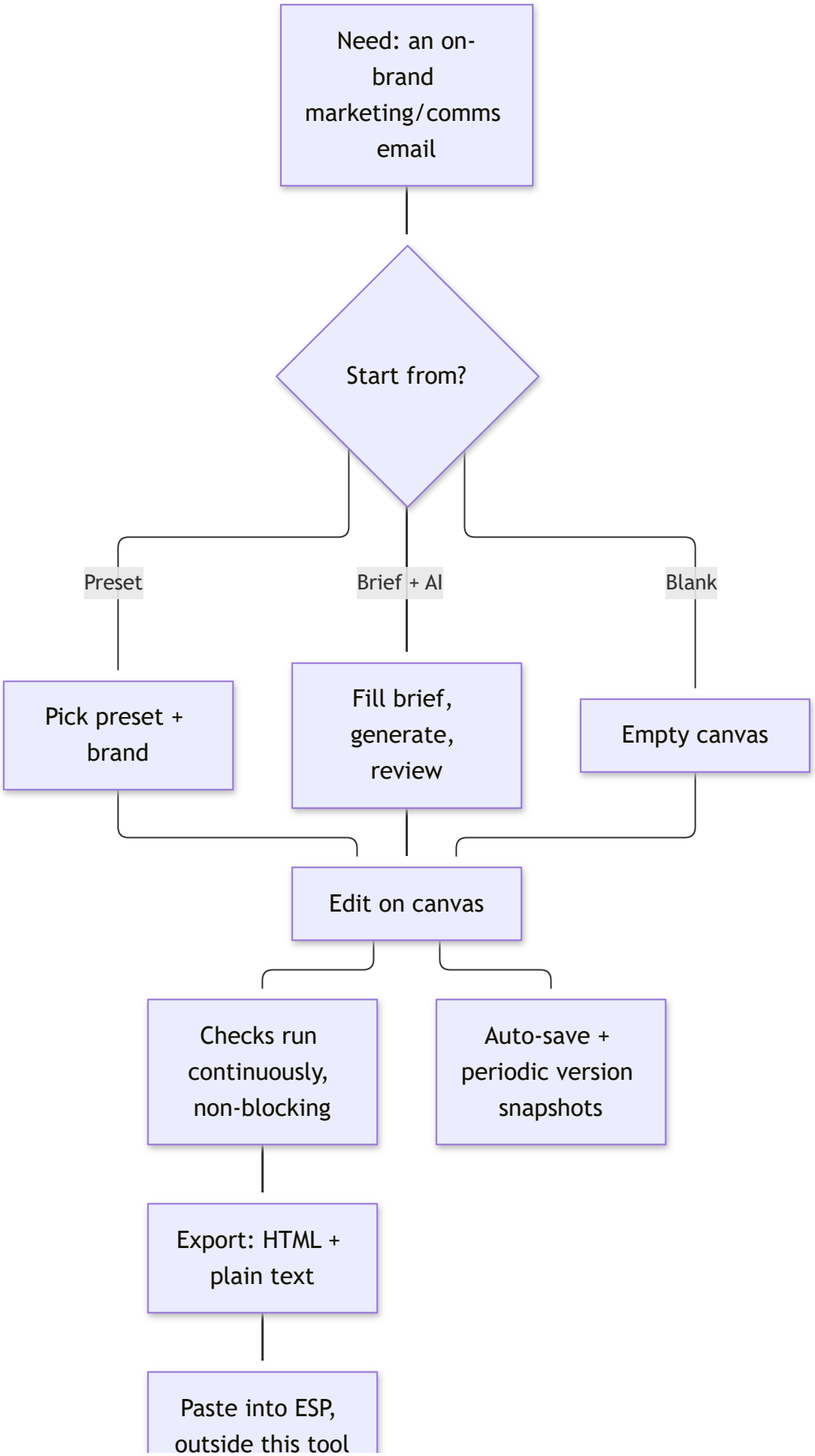
Can I send the email directly from this app? No. The app produces HTML and plain text; sending happens through whatever ESP the team already uses (SES, SendGrid, Mailchimp, HubSpot, etc.), entirely outside this tool.

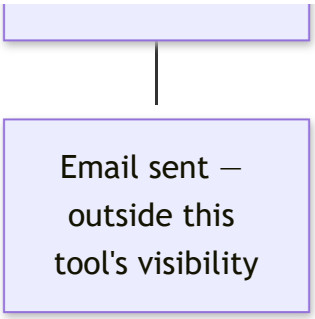
What if an AI-generated image never gets a real public URL? It will keep exporting as a visible placeholder image (and the Checks drawer will keep warning about it) until someone downloads the generated PNG, uploads it somewhere public, and pastes that https URL into the field — there is no image-hosting feature built in yet.

---

35. Business Process Diagrams

35.1 Overall business process

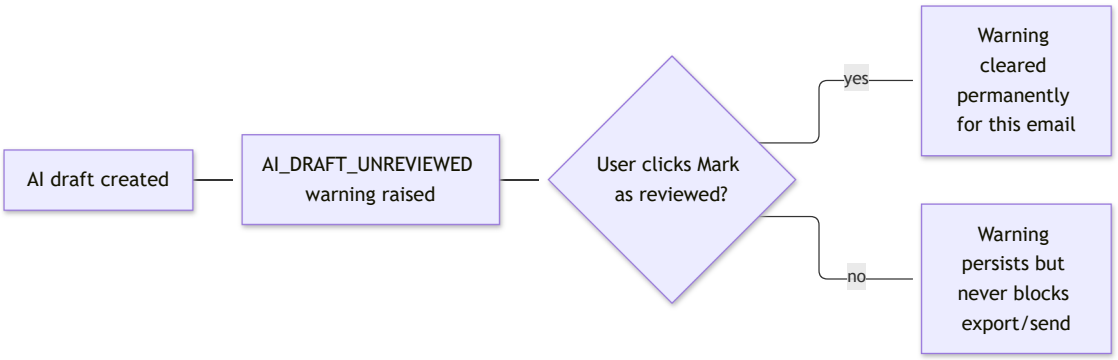

Need: an on-brand marketing/comms email
Start from?
Preset
Brief + AI
Blank
Pick preset + brand
Fill brief, generate, review
Empty canvas
Edit on canvas
Checks run continuously, non-blocking
Auto-save + periodic version snapshots
Export: HTML + plain text
Paste into ESP, outside this tool



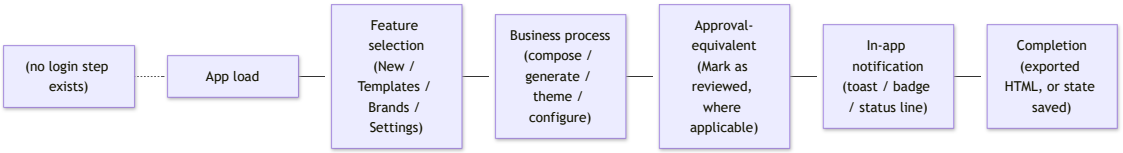
35.2 Department workflow

Not Found in Codebase / Not Applicable — a single department (marketing/communications) and a single role perform every step above; there is no cross-department handoff to diagram.

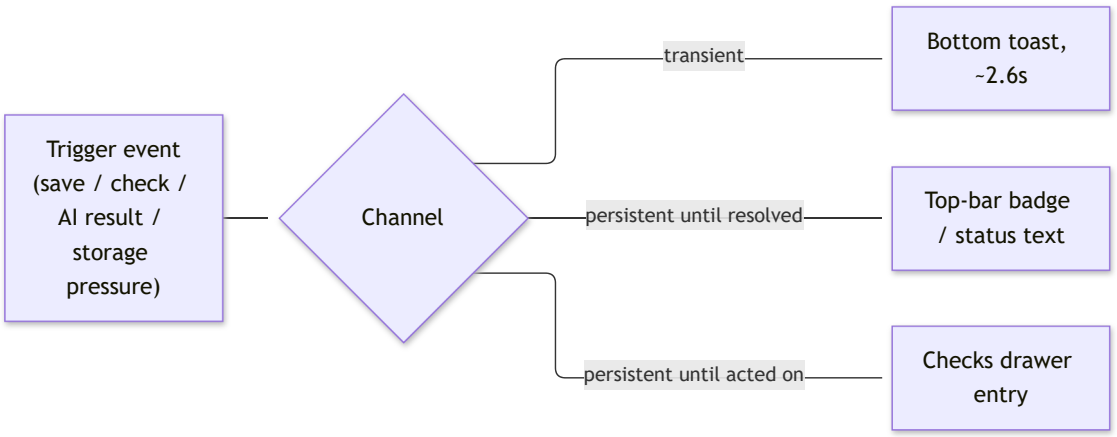
35.3 Approval workflow



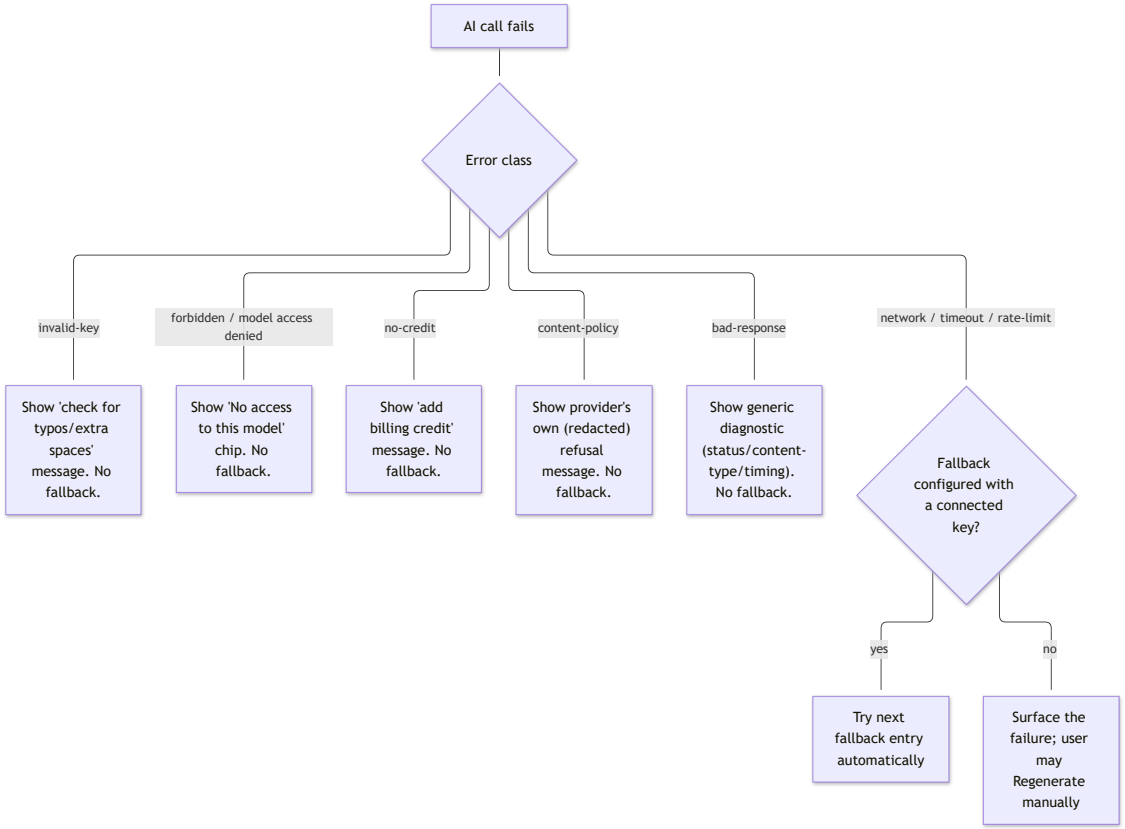
35.4 User journey (consolidated — see §12 for the six full variants)



35.5 Notification flow



35.6 Decision tree — "why did my AI call fail, and what happens next?"



35.7 Role interaction

Not Found in Codebase / Not Applicable — one role.

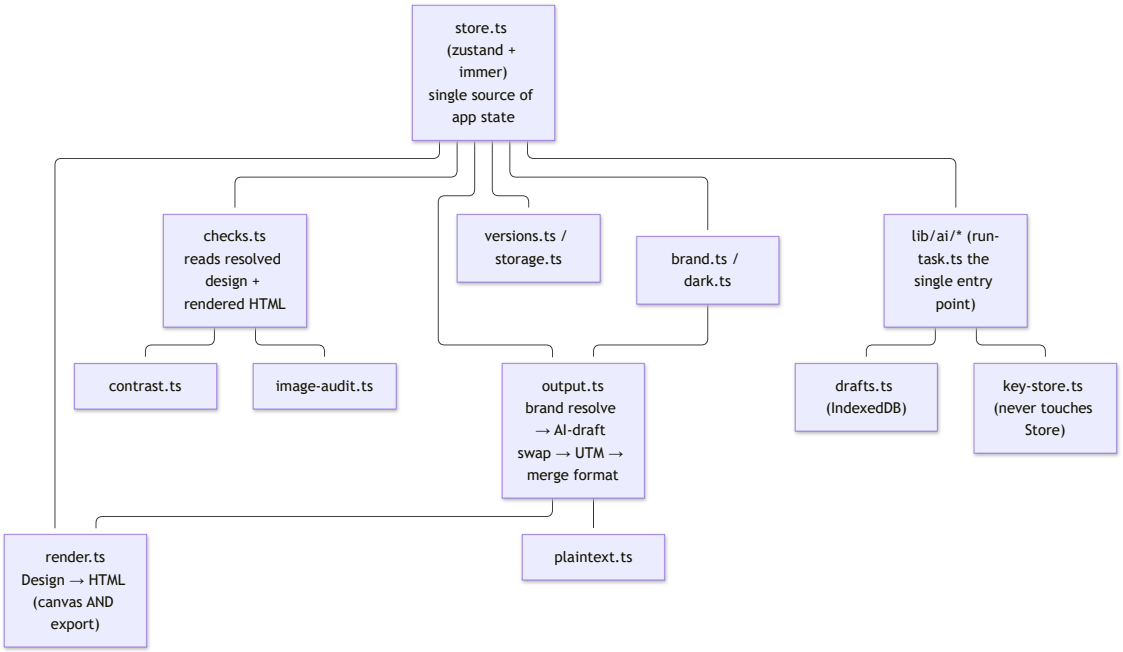
35.8 State transition

See §14 for the three state machines that exist (saved-email lifecycle, AI-brief-review lifecycle, per-issue Checks lifecycle).

35.9 Navigation flow

See §11.1.

35.10 Module interaction


store.ts (zustand + immer) single source of app state
checks.ts reads resolved design + rendered HTML
versions.ts / storage.ts
brand.ts / dark.ts
lib/ai/* (run-task.ts the single entry point)
contrast.ts
image-audit.ts
output.ts brand resolve → AI-draft swap → UTM → merge format
drafts.ts (IndexedDB)
key-store.ts (never touches Store)
render.ts Design → HTML (canvas AND export)
plaintext.ts

36. End-to-End Business Scenario

Scenario chosen: *"The MARCOM team sends a Diwali promotional email, drafted by AI, themed to the Vee Technologies brand, with an AI-generated hero image, checked for quality, and exported for their ESP."* This scenario deliberately exercises the two most business-significant workflows (6A and 5A) end-to-end, referencing the actual screens/modules/entities involved.

```
Business Need
  The team needs a promotional email about a Diwali sale, going out soon, and
  nobody has time to write a first draft from scratch or hunt for stock art.
  ↓
User Action (Screen: TopBar "New" → NewEmailModal → "Generate from brief" tab)
  The editor fills BriefForm: topic="Diwali sale on annual plans", audience=
  "existing customers", goal="promote", 3 key points, tone="bold", length=
  "medium", CTA left blank, language="English".
  ↓
Validation (BriefForm.tsx: briefError())
  Topic present; total characters under BRIEF_MAX_CHARS (4000); CTA URL
  field empty so its https-only rule doesn't apply.
  ↓
Decision Point (isTaskReady('text.brief'))
  The text.brief task already has a Connected OpenAI key from a prior
  session — no detour to Settings → AI is needed.
  ↓
Decision Point (hasTextConsent('openai'))
  First time for this provider → the data-use notice is shown: "Your brief
  and Vee Technologies's brand voice are sent to OpenAI to generate this
  draft. Nothing else — no API key, logo, footer or other saved content."
  The editor clicks "Agree and generate" (grantTextConsent('openai') fires).
  ↓
Processing (create-from-brief.ts + slots.ts + brief-prompt.ts + OpenAI
  Chat Completions API)
  goal="promote" auto-resolves to the "promotional" preset (preset-
  selection.ts). buildSlotSchema() walks that preset's real block tree and
  finds its AI-fillable fields (heading text, hero eyebrow/heading/subtext,
  button label, etc.) — none of them hero-card fields, because this preset's
  Hero layout isn't cards-2/cards-3. One strict-JSON-schema request is sent;
  the response is validated field-by-field by brief-validate.ts (binary
  accept/reject).
  ↓
Review (Screen: BriefReview)
  The editor sees 3 subject-line options, a preheader, and one suggested
  image prompt for the Hero's image slot. No bracket placeholders remain —
  the brief gave enough detail. The editor picks a subject line.
  ↓
Approval-equivalent action (button: "Create email")
  writeBriefDraftOntoDesign() writes every slot value onto a FRESH copy of
  the promotional preset's Design; design.body.title and .preheader are set;
  a BriefRecord {reviewed:false} is attached. startNewEmail() makes this the
  active document (Checks.ts: AI_DRAFT_UNREVIEWED now active).
  ↓
Processing (Screen: RightPanel → Hero block → "Generate with AI")
  The editor selects the Hero image field and opens the AI image panel,
  pastes in the AI-suggested prompt text (copied from BriefReview), picks
  aspect "16:9", style "Cinematic elegant", quality "high", and toggles
  "Brand colours" on. Send → one request to OpenAI's image endpoint
  (openai.ts) → response validated by magic bytes → stored as a DraftImage
  in IndexedDB with a fresh aiDraftId.
  ↓
Validation (AiImagePanel.tsx)
  Alt text is required before "Add to email" is enabled; the editor types a
  description. target.onUse() writes the Hero's imageSrc field's aiDraftId.
  ↓
```

```
Data Update (store.ts: refreshDrafts())
  The app's knownDrafts set now includes this id, so Checks correctly
  classifies it as AI_IMAGE_NOT_HOSTED (bytes present locally, no public
  URL yet) rather than AI_IMAGE_MISSING_DRAFT.
  ↓
Notification (Checks drawer, debounced 500ms)
  Two warnings now show: AI_DRAFT_UNREVIEWED and AI_IMAGE_NOT_HOSTED. Both
  are warnings, not errors — export remains fully enabled throughout.
  ↓
Remediation
  The editor reviews the generated copy against the brief, is satisfied,
  and clicks "Mark as reviewed" (clears AI_DRAFT_UNREVIEWED). They download
  the generated PNG, upload it to the team's existing image CDN (outside
  this app), and paste the resulting https URL into the Hero image field —
  which clears aiDraftId and resolves AI_IMAGE_NOT_HOSTED automatically on
  the next Checks run.
  ↓
Completion (Screen: ExportHtmlModal)
  Checks now shows 0 errors, 0 warnings (or only acceptable info-level
  notes). The editor opens Export, confirms the size (well under 90KB) and
  the UTM summary (UTM is off for this send, so 0/0), downloads the .html
  file, and pastes it into their ESP's campaign editor — sending itself
  happens entirely outside Newsletter Builder.
```

Entities touched: `Brand` (Vee Technologies, read-only in this scenario), `BriefFields`/`BriefDraft`/`BriefRecord`, `Design`/`SavedTemplate` (the new email, auto-saved along the way), `DraftImage` (IndexedDB), `Issue` (two warnings raised and resolved). **Screens touched:** NewEmailModal, BriefForm, BriefReview, RightPanel, AiImagePanel, ChecksPanel, ExportHtmlModal. **Modules touched:** 1A (brand), 1C (preset), 2A (export boundary), 3A (Checks), 5A (AI image), 6A (brief generation). **APIs touched:** OpenAI Chat Completions (text) and Images (image) endpoints, both called directly from the browser.

---

37. Traceability Matrix

Req ID	Business Objective	Module	Feature	Screen	Workflow	Business Rule(s)	Validation	API	DB Entity	User Role	Status
RTM-01	Brand consistency	Brand kit	1A	BrandsModal	§12.4	BR-01, BR-02, BR-03, BR-04	Name required	—	Brand	Editor	Shipped
RTM-02	Faster authoring	Canvas / Composite blocks	1B	Canvas, Right Panel	§7.1	§9.2 rules	Alt text required	—	Row/Column/Block	Editor	Shipped
RTM-03	Faster time-to-draft	Presets	1C	NewEmailModal	§7.1	BR-24, BR-25	—	—	Design (via preset.build)	Editor	Shipped
RTM-04	ESP-ready export	Export / merge tags / UTM	2A	Modals (Export)	§7.1	BR-19, BR-20, BR-21	CTA/URL rules (brief only)	—	BodySettings.mergeFormat/utm	Editor	Shipped
RTM-05	No lost work	Auto-save / versions / saved blocks	2B	TopBar, HistoryModal, Modals (Templates), LeftPanel (Saved tab)	§12.5, §14.1	BR-11, BR-12, BR-13, BR-25	Name required (rename)	—	SavedTemplate, VersionRecord, SavedBlock	Editor	Shipped

Req ID	Business Objective	Module	Feature	Screen	Workflow	Business Rule(s)	Validation	API	DB Entity	User Role	Status
RTM-06	Fewer broken images	Image library	2C-1	ImageLibrary, ImagePickerModal	§12.3 (partial)	BR-07, BR-17	URL scheme/shape validation	—	LibraryImage	Editor	Shipped
RTM-07	Fewer broken sends	Checks (quality engine)	3A	ChecksPanel	§7.3, §14.3	BR-05, BR-07	Size/contrast/link/subject rules	—	Issue (in-memory) + ignored-issues store	Editor	Shipped
RTM-08	Correct dark-mode rendering	Dark mode	3B	TopBar (scheme toggle), PreviewModal	§7.1 (export branch)	BR-22, BR-23	—	—	Brand.darkTokens	Editor	Shipped
RTM-09	On-brand imagery without a separate tool	AI image generation	5A	AiImagePanel	§12.3	BR-06, BR-08	Magic-byte validation; alt text required	OpenAI Images API	DraftImage (IndexedDB)	Editor	Shipped (no hosting → §29 gap)
RTM-10	Faster first drafts	AI brief generation	6A	NewEmailModal (Brief tab), BriefForm, BriefReview	§7.2, §12.2, §14.2, §36	BR-26, BR-27, BR-28	Topic required; 4000-char cap; CTA https	OpenAI Chat Completions API	BriefRecord	Editor	Shipped

Req ID	Business Objective	Module	Feature	Screen	Workflow	Business Rule(s)	Validation	API	DB Entity	User Role	Status
RTM-11	Resilient, controllable AI usage	Per-task model routing	6B	AiSection	§13	BR-29, BR-30	—	OpenAI (GET /v1/models, chat, images)	TaskRoutes	Editor	Shipped
RTM-12	Consolidated configuration	Settings hub / Profile / Data governance	7A	SettingsHub + 7 sections	§12.6, §9.12	BR-14, BR-15, BR-16	Display name ≤40 chars; typed "DELETE" gate	—	CurrentUser, EmailDefaults, BackupData	Editor	Shipped

38. Production Readiness (Business Perspective)

Dimension	Assessment
Business completeness	High for its stated scope (design → check → export). Zero for anything it explicitly never claimed to do (sending, hosting, multi-user).
Workflow completeness	The AI-brief and AI-image workflows are fully wired end-to-end, including a human-review step, but both terminate in a **manual** handoff (review click; manual image rehosting) rather than a fully closed loop.
User experience	Consistently designed around "never block, always explain" (Checks, storage-full messaging, AI error redaction) — a strong, deliberate UX principle applied uniformly across very different features.
Operational readiness	Weak outside the browser: no backup is automatic, no monitoring/telemetry exists, and the "production" environment is literally every individual user's own browser profile with no central ops visibility.
Maintainability	Strong: strict TypeScript config, 39 test files, documented architectural rules (`docs/PROGRESS.md`) with explicit rationale for nearly every non-obvious decision. **Gap:** no dedicated lint script.
Extensibility	Strong in the areas the team clearly anticipated extending (AI tasks, slot derivation, brand permission seams); weaker where no seam was built (image hosting, a second AI provider, roles).
Documentation quality	Unusually high for an internal tool — `docs/PROGRESS.md` documents business rationale, known gaps, and test coverage per feature, which is what made this BRD possible to produce with very few "Not Found" gaps.
Business risks	Concentrated in exactly two places: (1) total reliance on one browser's local storage with no automatic off-machine backup, and (2) uncapped, unmonitored direct AI billing exposure per user. Both are named explicitly in this document (§28) rather than hidden.
Training requirements	**Inference:** Low for the core editing flow (visual, drag-and-drop); Medium for the AI features (requires understanding BYOK, consent, and the review step is advisory not enforced); Low for Settings (one hub, clearly labeled).

Dimension	Assessment
Overall score	**68 / 100** (Inference — a qualitative judgment, not a measured score). The product is a complete, well-engineered, honestly-documented tool for exactly the single-user, single-browser scope it set out to cover — the score is held back entirely by the absence of anything beyond that scope (no sync, no hosting, no sending, no cost controls, no monitoring), all of which the product itself is explicit and unapologetic about not providing yet.

Strengths: rigorous, test-verified business-rule discipline (resolve-at-the-boundary, never-block-only-warn, no-double-bill-retry, never-accuse-before-looking); genuinely defense-in-depth AI-key security engineering for a client-only app; an unusually honest internal documentation trail that distinguishes shipped behavior from known gaps feature-by-feature.

Weaknesses: no automatic backup/sync mechanism for a product whose entire data model is local-only; no AI cost visibility or caps; the human-review gate on AI content is advisory, not enforced; no accessibility/localization investment in the builder UI itself (only in its output); no CI evidence found in the reviewed scope.

Recommendations: (1) prioritize image hosting and/or a Supabase-backed persistence layer (both already named in the README's own roadmap) to close the two largest business-continuity risks at once; (2) add a lightweight AI-usage cost/counter display, even client-side-estimated, given zero visibility exists today; (3) consider whether the unreviewed-AI-content warning should be strengthened (e.g., a one-click "I reviewed this" requirement before Export, rather than purely informational) given it currently carries no teeth.

---

39. Final Business Knowledge Validation

"If You Read This Entire BRD"

You now understand:

✓ **Business objectives** — a single internal marketing/communications team producing on-brand, inbox-safe HTML emails, increasingly assisted by AI, entirely without a backend or a login.

✓ **Stakeholders** — effectively one functional role (the Editor), with deliberate, unbuilt seams for a future multi-user system, and OpenAI as the only external party the product talks to.

✓ **User roles** — exactly one, unrestricted, with no RBAC anywhere; "Profile" is a local label, not an account.

✓ **Business processes** — compose → (optionally AI-draft, with mandatory-in-spirit-but-not-enforced human review) → check (always advisory) → export → hand off manually to an ESP.

✓ **Functional requirements** — 12 shipped features (1A through the 6B amendment) spanning brand theming, composite content blocks, presets, merge/UTM/plain-text export, auto-save/versioning/reuse, an image library, a non-blocking quality engine, dark mode, two AI pipelines with bring-your-own-key security, per-task model routing with fallback, and a consolidated Settings/data-governance hub.

✓ **Business rules** — 30 named rules (§13), the most load-bearing being "resolve at the boundary," "never block, only warn," "no retry that could double-bill," "never accuse before looking," and "API keys never enter serializable state" — each enforced by dedicated tests, not just convention.

✓ **Validation rules** — exhaustively cataloged in §17, from image-URL scheme checks to WCAG contrast math to the AI brief's strict-JSON-schema acceptance gate.

✓ **User journeys** — six complete, screen-by-screen paths (§12) covering first use, AI-brief drafting, AI-image generation, brand management, mistake recovery, and data portability.

✓ **Workflow states** — the saved-email lifecycle, the AI-brief review lifecycle, and the per-issue Checks lifecycle (§14) — notably, the absence of any multi-party approval state machine, which this document states plainly rather than inventing one.

✓ **Navigation** — a single-screen SPA with no router, no URLs, and no role-based menu variation; every "screen" is a modal, overlay, or panel-swap (§10, §11).

✓ **Modules** — 15 modules (§8), each with a clear single business purpose and no overlapping ownership.

✓ **Features** — full Given/When/Then-ready documentation for all 12 (§9, §31).

✓ **Reports** — effectively none in the BI sense; only live, in-context status readouts (§19).

✓ **Dashboards** — effectively none; a single Checks-severity badge is the closest analogue (§20).

✓ **Notifications** — entirely in-app and ephemeral; no email/SMS/push channel exists (§18).

✓ **Integrations** — exactly one real one (OpenAI, direct from the browser, BYO key), and one conspicuously absent-by-design one (no ESP/sending integration) (§21–22).

✓ **Security** — no authentication exists at all; the one genuinely sensitive artifact (the AI API key) is handled with real, test-verified defense-in-depth despite the absence of any backend to enforce it server-side (§23).

✓ **Data requirements** — a clean, well-normalized local data model (§15–16) with explicit, tested migration rules for every schema change in the product's history, and no server-side counterpart of any kind.

✓ **Traceability** — every shipped feature mapped end-to-end from business objective through screen, rule, validation, API, and data entity to its status (§37).

✓ **Risks** — concentrated and named plainly: no automatic off-device backup, uncapped direct AI billing exposure, and an advisory-only human-review gate on AI content (§28).

✓ **Future roadmap** — image hosting and a Supabase-backed persistence layer are the two highest-leverage next steps, both already named by the product's own maintainers, not invented for this document (§30).

You can now confidently participate in product discussions, requirement gathering, QA validation, stakeholder meetings, backlog refinement, feature-enhancement planning, and future product development for Newsletter Builder **without reverse-engineering the code yourself** — and, just as importantly, without mistaking any of its deliberate current-state boundaries (no login, no sending, no hosting, no cross-device sync) for defects rather than documented, intentional scope.

Appendix A: Verification Notes

Every **Inference** and **Not Found in Codebase** statement across `BRD-01.md`–`BRD-04.md`, listed by section, with the reasoning behind each tag. Anything not listed here is a **Fact** — stated directly in source code, code comments, tests, or `docs/PROGRESS.md`. Nothing in the BRD was guessed; where the codebase was silent, this document says so rather than inventing an answer.

Part 1 (BRD-01.md)

§1 Executive Summary

- **Inference** — Target audience is a small team sharing a few brand kits, not a multi-tenant SaaS audience. *Reasoning:* drawn from the per-browser, no-roles, export/import-as-sharing architecture; no explicit target-market statement exists in the code.
- **Inference** — Expected business outcomes (faster production, brand consistency, fewer broken sends, lower skill floor). *Reasoning:* the codebase implements the mechanisms that would produce these outcomes but contains no analytics/telemetry to measure them, and no outcome statement exists.

§3 Business Objectives

- **Inference** — All "primary" and "secondary" objectives. *Reasoning:* no OKR/objectives document exists; objectives were derived from the pattern of what each shipped feature was built to solve, per `docs/PROGRESS.md`'s own framing.
- **Not Found in Codebase** — Business KPIs. *Reasoning:* exhaustive — no event tracking, error-reporting SDK, or usage-metrics code exists anywhere in `src/`.
- **Not Found in Codebase** — A formal, business-level success-criteria document. *Reasoning:* only implementation-level, test-verified acceptance bars exist per feature (used instead in §31).
- **Not Found in Codebase** — Expected ROI / cost model / pricing. *Reasoning:* no cost model exists; confirmed by exhaustive grep across `src/lib/ai/*` for cost/billing/quota/pricing terms (per the `ai-features.md` research pass) finding no usage-cap or cost-estimator code.

§4 Stakeholders

- **Not Found in Codebase** — A human stakeholder list, org chart, or RACI document. *Reasoning:* none exists; the table presented is reconstructed.
- **Inference** — The entire stakeholder table (Marketing/MARCOM user, Brand owner, future admin role, OpenAI-as-stakeholder) and the RACI-style matrix. *Reasoning:* reconstructed from the single named user base ("Vee MARCOM team," a Fact from `docs/PROGRESS.md`) plus the product's own future-proofing code seams (`createdBy`/`updatedBy`, `canEditBrand`, `getAccessibleBrands`), since no stakeholder document exists to source this from directly.

§5 User Roles

- **Not Found in Codebase** — Any role beyond a single, undifferentiated user; any viewer-only/approver-only/admin-only role; any "my emails vs. someone else's" concept. *Reasoning:* `user.ts`'s own header comment states "no login, no roles"; confirmed by exhaustive grep for `auth|login|password|session` across `src/` finding no app-level authentication anywhere.
- **Inference** — The label "Editor" for the one functional role. *Reasoning:* the code never names this role; "Editor" is a label of convenience for this document.

§6 Functional Scope

- No new Inference/Not-Found tags beyond those already listed above; the table restates Facts from features documented in §9.

§7 Complete Business Process

- **Not Found in Codebase** — Any second-person/hierarchical approval anywhere in the product. *Reasoning:* the only approval-like gate (`AI_DRAFT_UNREVIEWED` → "Mark as reviewed") is clearable by the same single role that generated the draft; no multi-party sign-off exists.

§8 Module Documentation / §9 Feature Documentation

- No additional Inference/Not-Found tags — restates Facts from `docs/PROGRESS.md` and source, with business-owner/user columns using the single Editor role established in §5.

§10 Screen Documentation

- **Fact, stated explicitly because it would otherwise look like an omission:** there is no URL router — confirmed via `App.tsx`'s `ModalName` union/component-state approach, with no `react-router` or history-API usage found. Role-based navigation, hidden-but-reachable pages, and redirects are **Not Found in Codebase** for the same reason (no routing layer exists to have any of those).

Part 2 (BRD-02.md)

---

§11 Navigation Flow

- **Not Found in Codebase** — Role-based navigation. *Reasoning:* single role (§5); no conditional menu rendering by identity exists.
- **Not Found in Codebase** — Hidden pages reachable by URL. *Reasoning:* no URL surface exists at all.
- **Not Found in Codebase** — Redirects. *Reasoning:* no routing layer exists to redirect within.

§13 Business Rules

- **Inference** — The "Priority" column (Critical/High/Medium) on every rule (BR-01 through BR-30). *Reasoning:* the code and `docs/PROGRESS.md` state *that* each rule exists and *why*, but never assign it a formal priority level; priority here is a judgment call based on how central each rule is to the product's core trust/safety guarantees.

§14 Workflow States

- **Not Found in Codebase** — Any multi-party approval workflow with states like "Pending Approval"/"Rejected." *Reasoning:* no such state machine exists anywhere in the product; the two state machines documented (saved-email lifecycle, AI-brief review lifecycle) are the only ones that exist.
- **Not Found in Codebase** — Any reversion of the AI-brief "Reviewed" flag after a later edit. *Reasoning:* confirmed no such reset logic exists in `store.ts`'s `markReviewed()`/related code.

§15 Data Requirements

- **Inference** — Classification of entities into "master data" vs. "transactional data." *Reasoning:* the codebase has no such taxonomy; this is a standard BRD categorization applied by this document's author based on how each entity is created/updated/shared in practice.
- **Inference** — "Unbounded in count" for templates, brands, saved blocks, and images. *Reasoning:* confirmed by the absence of any count cap in the quota-recovery code path (`writeSafe()`'s pruning logic only touches version history) — the absence of a limit is itself the finding, but "no one will ever hit a practical ceiling" is not claimed, only "no code-enforced cap exists."

§16 Business Data Dictionary

- **Inference** — The "Business owner" column in the entity-ownership summary table. *Reasoning:* since there is only one role, "owner" is inferred to be "the editing user" for every entity; no explicit ownership model exists in code beyond the `createdBy`/`updatedBy` attribution labels.

§18 Notifications

- **Not Found in Codebase** — Any outbound notification channel (email/SMS/push), scheduling, retry, or escalation mechanism. *Reasoning:* exhaustive — all "notifications" found are in-app, ephemeral UI (toasts, status text, badges).

§19 Reports

- **Not Found in Codebase** — Any report in the traditional BI sense (saved definitions, filters/columns, CSV export, scheduling). *Reasoning:* none exists; the table presented lists the closest functional analogues (storage-usage readout, Checks summary, backup-import summary, UTM reconciliation line) under an explicitly labeled Inference heading.

§20 Dashboards

- **Not Found in Codebase** — Any dashboard (widgets, charts, KPIs, role-based metric views). *Reasoning:* none exists anywhere in `src/`; the Checks severity badge is presented as the only functional analogue, explicitly labeled as such.

Part 3 (BRD-03.md)

---

§22 External Systems

- **Not Found in Codebase** — Identity Provider/SSO, ERP/CRM/HRMS/Payroll, Email/SMS gateway integrations. *Reasoning:* none referenced anywhere in the codebase; the only real external system integrated is OpenAI (§21, a Fact).

§23 Security Requirements

- **Not Found in Codebase** — Authentication of any kind (login, session, password, OAuth, JWT, server-side identity). *Reasoning:* `user.ts`'s own comment plus an exhaustive grep for `auth|login|password|session` across `src/`, every hit being AI-provider-key or in-tab AI-session terminology, never app authentication.
- **Not Found in Codebase** — Functioning authorization/RBAC. *Reasoning:* `canEditBrand()`/`getAccessibleBrands()` are documented in-code as future seams that always currently return "true"/"everything."
- **Not Found in Codebase** — Role management, formal audit log, compliance-framework reference (GDPR/SOC2/HIPAA/etc.), password policy, app-level session timeout/expiry. *Reasoning:* none of these concepts exist anywhere in the reviewed source.
- **Fact, stated for completeness:** the one real approval-like gate ("Mark as reviewed") performed by the same single role — not a second-person approval, which is **Not Found in Codebase**.

§24 Audit Trail

- **Not Found in Codebase** — A diff-level changelog between versions (only full snapshots exist); a record of who approved what; a log of actual AI request/response payloads sent to OpenAI (only the brief's own input fields are stored, not the raw exchange); any login/access log; any export/send log.
- **Not Found in Codebase** — Any compliance requirement/regulatory framework reference.

§25 Non-Functional Requirements

- **Not Found in Codebase** — Any load-testing artifact or performance budget beyond the email-size Checks rules (which govern output quality, not app performance).
- **Inference** — "Availability equals the user's own browser plus OpenAI's reachability." *Reasoning:* follows necessarily from the fully client-side architecture; not a statement found verbatim in code.

- **Inference** — Templates/brands/images/saved blocks are "unbounded in count" with no automated pruning (same finding as §15, restated in the NFR context).
- **Not Found in Codebase** — Any accessibility statement/audit for the builder UI itself (as opposed to the *output email's* accessibility, which is extensively engineered and a Fact).
- **Not Found in Codebase** — Localization of the application UI itself (only AI-generated *content* supports multiple languages).
- **Not Found in Codebase** — Any telemetry, error reporting, or usage analytics (monitoring).

§26 Assumptions

- Every row in this section is explicitly an **Inference** by definition (that is the section's purpose) — each row states its own Fact/Inference blend in its "Status" column; none claim to be pure Fact beyond the quoted "Vee MARCOM" framing (#1) and the "no sending" README quote (#4).

§27 Constraints / §28 Risks

- No new Inference/Not-Found tags — these sections restate Facts already established (BYOK architecture, CORS header allow-list, storage limits) and apply standard risk/constraint framing to them, which is itself an ordinary analytical step rather than a claim requiring separate verification.

§29 Gap Analysis

- **Not Found in Codebase** — Any CI configuration file in the directories reviewed for this document; whether `npm test`/`npm run build` run automatically on push is explicitly marked unresolved within this review's scope (a narrower, honest claim than asserting either way).

§30 Future Enhancements

- **Fact** — The specific enhancements named (image hosting, Supabase persistence, test-email sending, merge-tag picker groups, countdown-timer block, template-gallery thumbnails, AI "generate section") are a direct, near-verbatim restatement of the README's own "Ideas for next versions" list.
- **Inference** — The "Priority" column and the two enhancements not in that README list (a second AI provider; AI usage/cost visibility). *Reasoning:* priority is a judgment call; the two additional items are drawn directly from gaps/risks already identified elsewhere in this document (§28/§29), not from the README.

Part 4 (BRD-04.md)

---

§31 Acceptance Criteria / §32 User Stories

- **Inference** — Every Given/When/Then block and every user story. *Reasoning:* no formal acceptance-criteria or user-story document exists in the repository; all are phrased from the behavior described

in `docs/PROGRESS.md` and confirmed directly against source, not copied from any existing artifact.

§35 Business Process Diagrams

- **Not Found in Codebase / Not Applicable** — §35.2 "Department workflow" and §35.7 "Role interaction." *Reasoning:* a single department and a single role perform every step in this product; there is no cross-department or cross-role handoff to diagram, so the template section is marked not applicable rather than populated with an invented diagram.

§36 End-to-End Business Scenario

- **Fact, with one explicit narrative choice:** the scenario and every screen/module/entity/API it names are Facts drawn from earlier sections; the specific combination of inputs (e.g., the exact brief wording used) is illustrative narrative built on those Facts, not a literal transcript from the codebase or its tests.

§37 Traceability Matrix

- **Fact** — Every row's Module/Feature/Screen/Workflow/Business-Rule/API/Entity columns map directly to content established earlier in the BRD. The "Status" column ("Shipped") reflects `docs/PROGRESS.md`'s own "done" markers for each feature.

§38 Production Readiness

- **Inference** — Every assessment in the "Assessment" column, and especially the numeric **"68/100"** overall score. *Reasoning:* explicitly and repeatedly flagged in the BRD text itself as "a qualitative judgment, not a measured score" — no scoring rubric, benchmark, or production-readiness audit exists in the codebase; this is this document's own synthesis, provided because the BRD prompt asked for one, and should be read as an opinion grounded in the Facts above it rather than a Fact itself.
- **Inference** — "Strengths," "Weaknesses," and "Recommendations." *Reasoning:* same basis as the score — qualitative synthesis, not sourced from any in-repo document.

§39 Final Business Knowledge Validation

- No new tags — this section is a summary/recap of claims already tagged earlier in the document; it introduces no new factual, inferred, or absent claims of its own.

Summary count

Tag	Approximate count across all 4 parts
Inference	~45 distinct statements/sections

Tag	Approximate count across all 4 parts
Not Found in Codebase	~30 distinct statements/sections

The heaviest concentrations of both tags are, unsurprisingly, in the sections a traditional enterprise BRD template assumes exist for any business application — **Stakeholders (§4)**, **Security Requirements (§23)**, **Audit Trail (§24)**, **Reports (§19)**, and **Dashboards (§20)** — because this product is a single-user, single-browser, no-login client application, not a multi-tenant enterprise system. That mismatch between the template's assumptions and the product's actual architecture is the single most important fact a reader of this BRD should take away, and it is the reason this verification log exists.